

Omniverse Base Workshop 1

KUKA X IPI

- 2x 4h Course with examples to
 - Learn the Basics in Omniverse, USD & Isaac Sim
 - Enable you to do Robotic model preparation & Simulation
 - Introduction to IsaacLab

Trainer Introduction



Fabian Fichtl



Julian Zürn



Institut für Produktion und Informatik

Technologietransferzentrum der Hochschule Kempten

- anwendungsorientierte Forschungseinrichtung
- F&E-Projekte im Kontext Digitalisierung in der Produktion
- Seit 2021 mit 30 Mitarbeitern
- Technologietransfer und Startup Inkubator
- VIBN und Industrial Metaverse in der Lehre





Introduction round

Please tell a few sentences about yourself,
what you expect from this workshop and
what you want to use Omniverse for

Agenda Tag 1:

- Introduction Metaverse
- Omniverse basics:
Nucleus | Data Formats | PhysX | extensions | Stage
- 5 min Pause
- Hands on: Scene manipulation, Kinematics and Assets
 - UI Manipulation Navigation, Manipulation (property window)
 - Mass & Mesh manipulation
 - First Robot
- 10 min Pause
- Import and Tune a Robot
- 5 min Pause
- First Robot Control (LULA test widget)
 - Adding Sensors and Kameras

Agenda Tag 2:

- Debugging Tools:
 - Console, VS Code, UI
- Scene Architecture
 - Build your Scenario: Environment, Robot, Gripper, Pick & Place
- Motion Planning & Motion Learning
- Materials
- Movie Capture Tool
- Information Channels | Further Courses



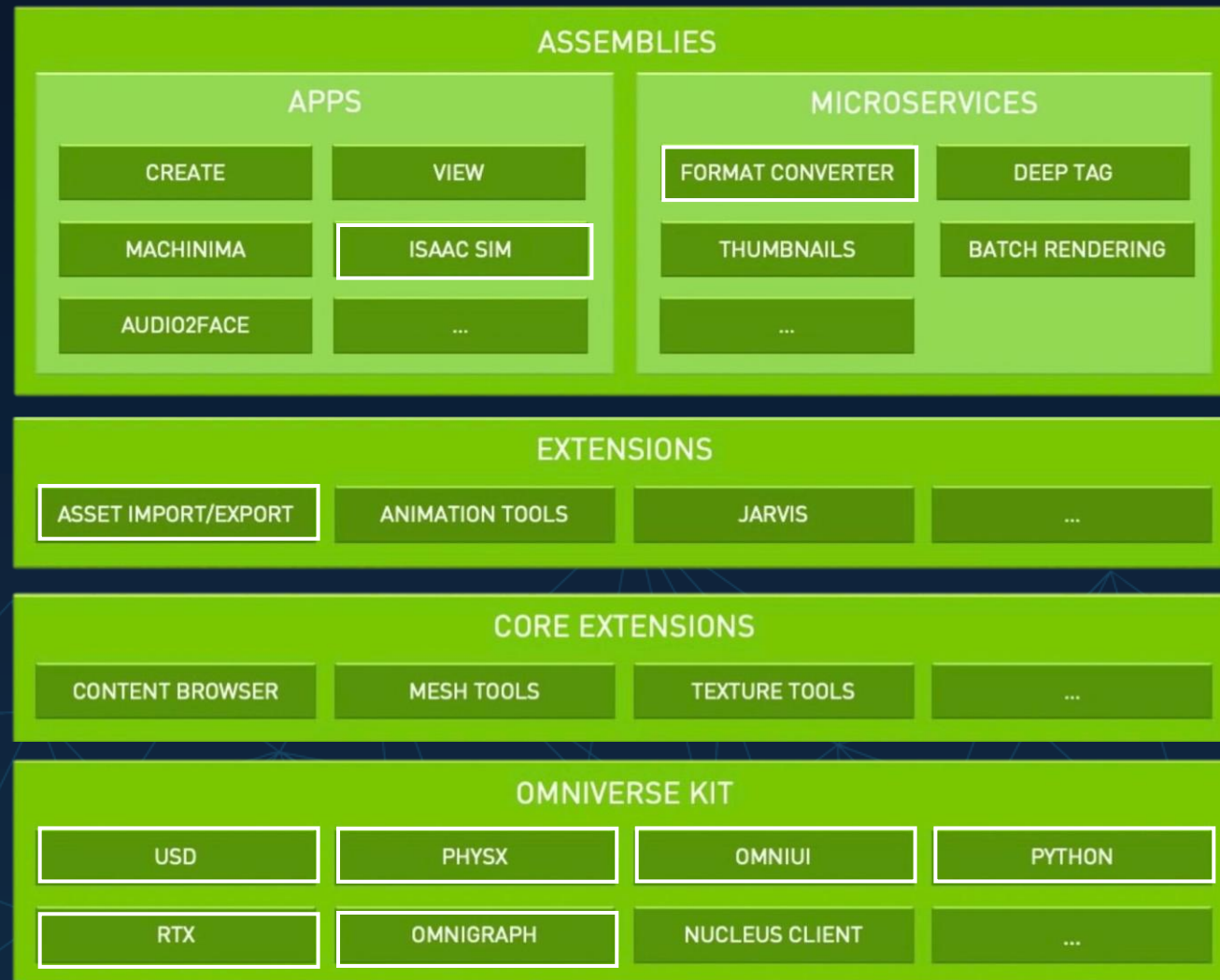
Introduction to Omniverse

Concept: Industrial Metaverse

- Virtual Environment
- Real-time Interaction and Collaboration
- Purpose can span the Design, Simulation, Observation or Replay of arbitrary Objects and Processes in any Industry
- The *NVIDIA Omniverse* platform is optimized to run 3D physics Simulations in real-time on *NVIDIA RTX* Hardware

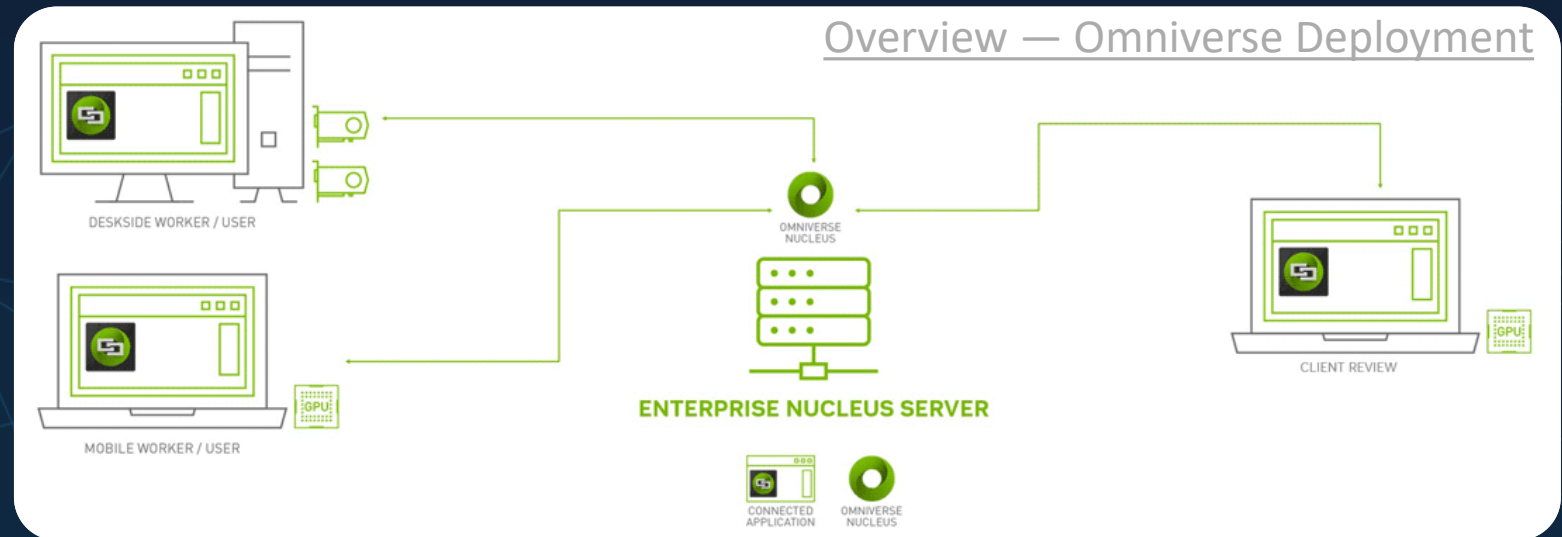


Omniverse Architecture



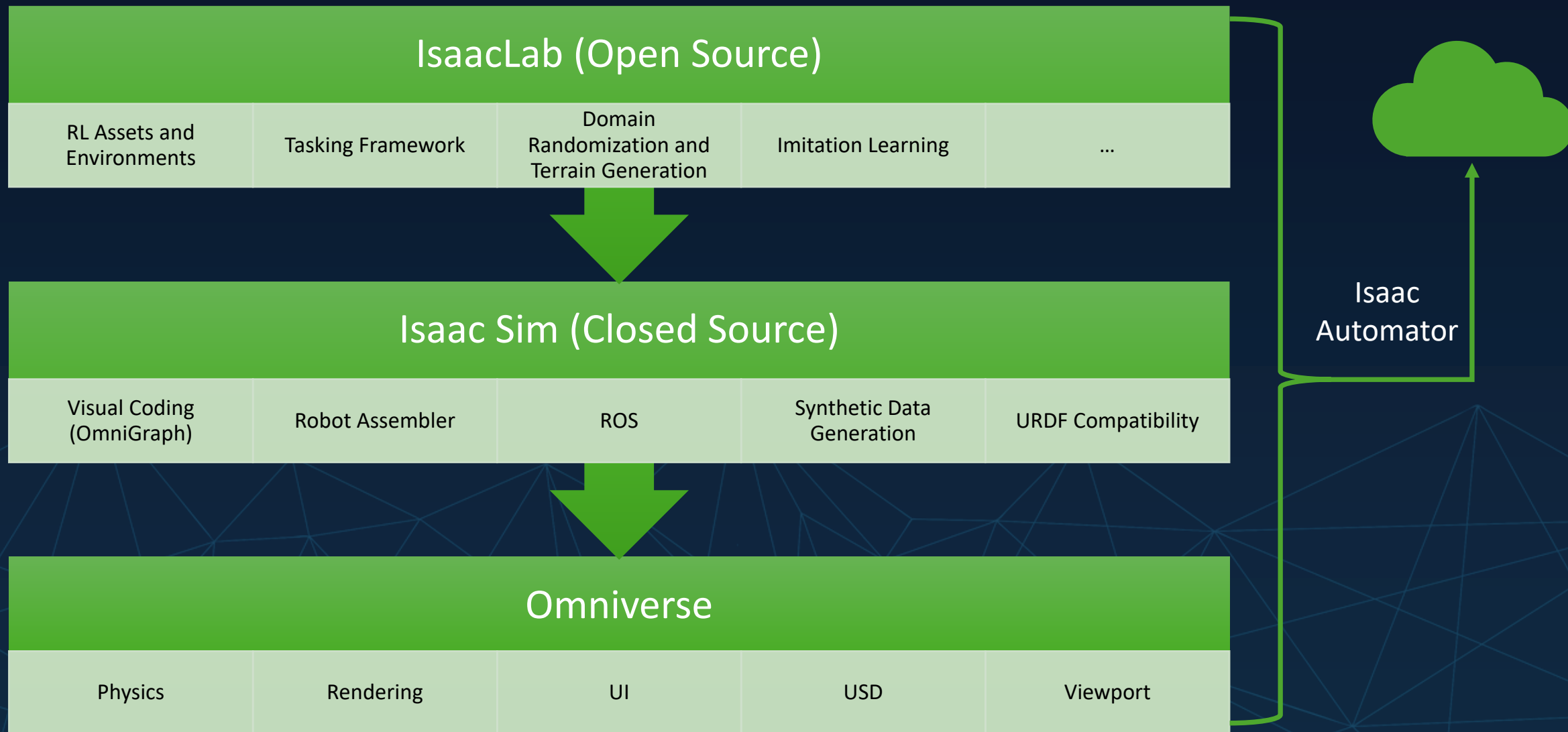
Nucleus – The Omniverse Database

- Enables Real-time Collaboration
- Comprehensive User/Permission Management
- Caching Mechanism for large Assets
- Wide API and Script Integration
- Enterprise Scalability





NVIDIA Isaac Ecosystem





Universal scene description



Interacting with USD (in Omniverse scene)

- UI Main Capabilities

- Viewport:
 - Scene
- Property Window:
 - Prim* Properties
- Omnigraph:
 - Process Logic
 - ROS Controller
- Various extensions

- API Main Capabilities

- Python interface
- Robot Controllers
- URDF import
- Create the scene for IsaacLab
- Configure the policy
- Start training

Robot Data formats

	URDF	SRDF	XRDF
Description Language	XML	XML	YAML
Purpose	Define Geometries and Kinematic Joints	Complements URDF Data with Semantic Grouping	Complements URDF Data with Kinematic Description
Collision Handling	Simple Shapes and Meshes	Collision Matrix (specifies handling for link pairs)	Collision Spheres along Robot Links
Joint Representation	Varying Types and Parameters	Defines Groups for Motion Planning	Advanced Configuration including Acceleration and Jerk Limits
Handling	Core Format in ROS and converted to USD upon import in Isaac Sim	ROS Moveit Assistant to generate Motion Plans	NVIDIA CuMotion and Lula to generate Motion Plans

Manual Modelling Phases Robot Simulation

1. Import URDF and convert to USD
2. Config Joint drives -> Test
3. Add Gripper/ Cameras
4. Add details to the stage of:
 1. URDF for ROS and RL
 2. SRDF for Moveit
 3. XRDF for Cumotion and LULA / RMPflow

NVIDIA PhysX

- The PhysX world comprises a collection of Scenes, each containing objects called Actors;
- Each Scene defines its own reference frame encompassing all of space and time;
- Actors in different Scenes do not interact with each other;
- Characters and vehicles are complex specialized objects made from Actors;
- Actors have a physical state: position and orientation, velocity or momentum, energy, etc;
- Actor physical state may evolve over time due to applied forces, constraints such as joints or contacts, and interactions between Actors.



Omniverse Stage



Until now:

- We reviewed necessary Omniverse Basics and Resources

5 min Break

To be continued:

- Practical session:
 - Extensions installation
 - Scene Manipulation
 - Robot import + tuning



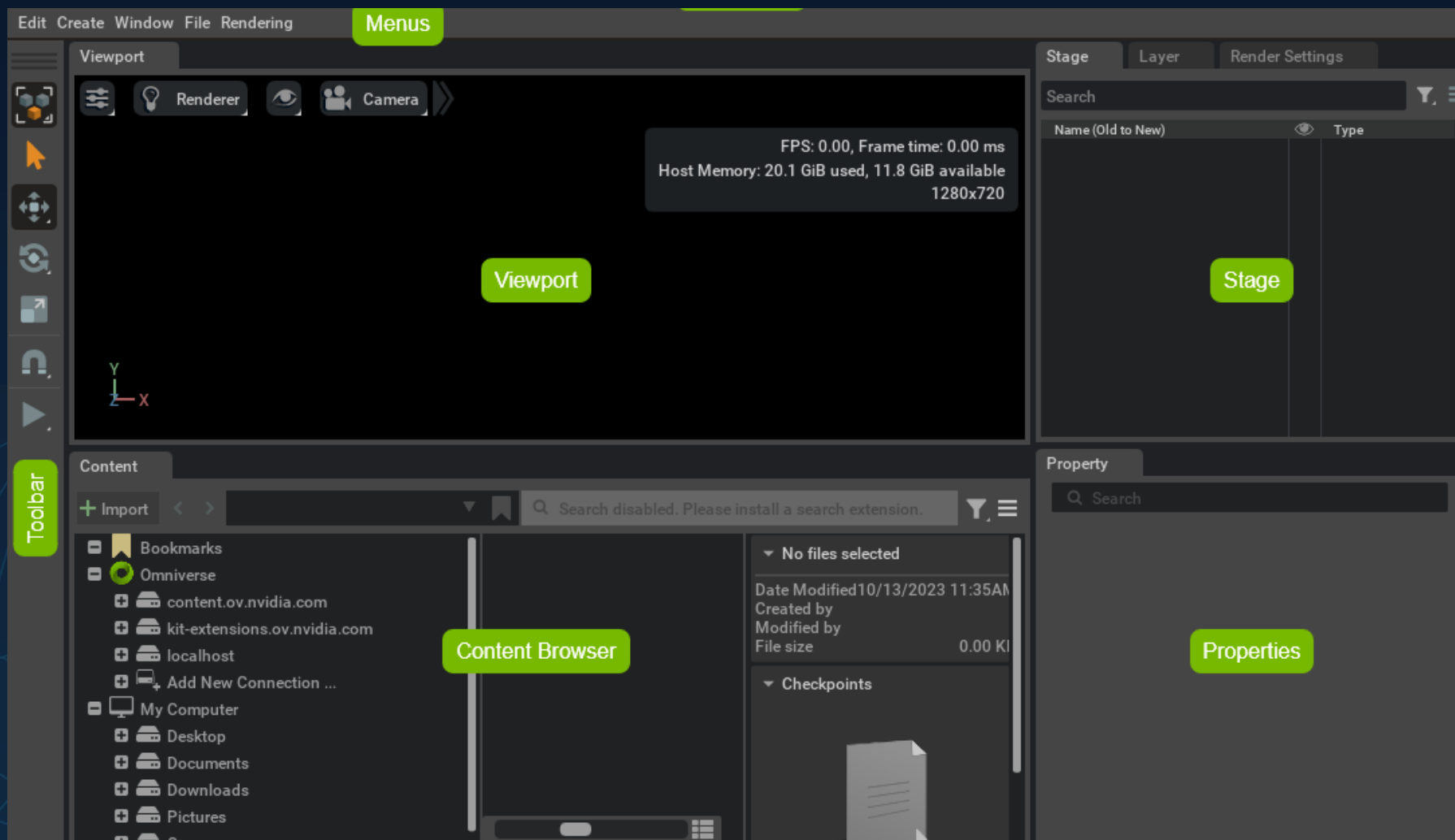
Hands on!

Getting started:

- Start Omniverse Isaac Sim on the Machine
 - Please give feedback if your application is loaded



Standard Layout

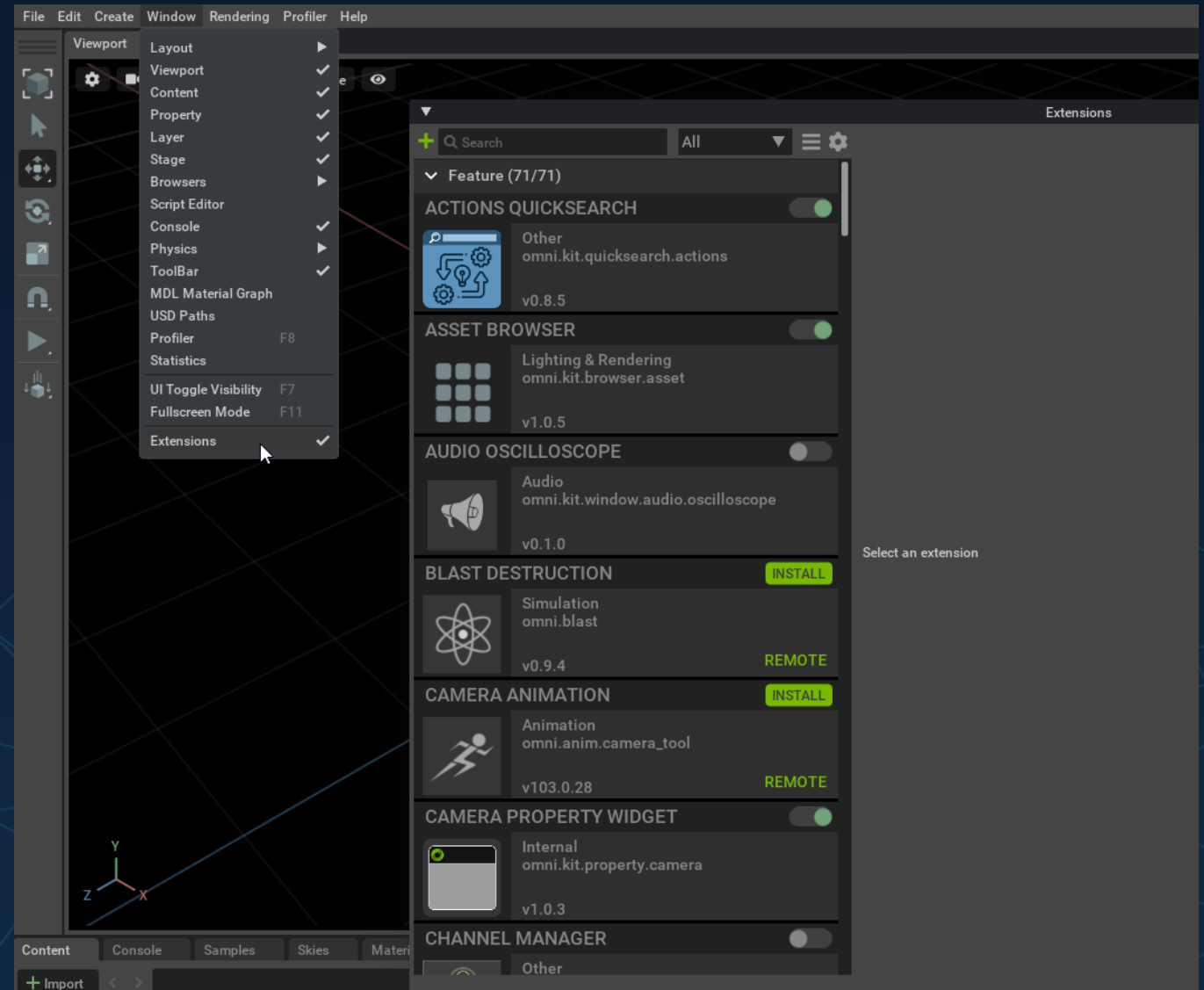


How-to: Viewport Navigation

- Orbit: Left mouse btn. + ALT
- Look: Right mouse btn. (RMB)
- Fly/ Walk: Hold RMB; *WASD* / Q up / E down
- Adjust Speed: Hold RMB; Scroll
- Jump to Selection: F

Scene Prep tools (extensions)

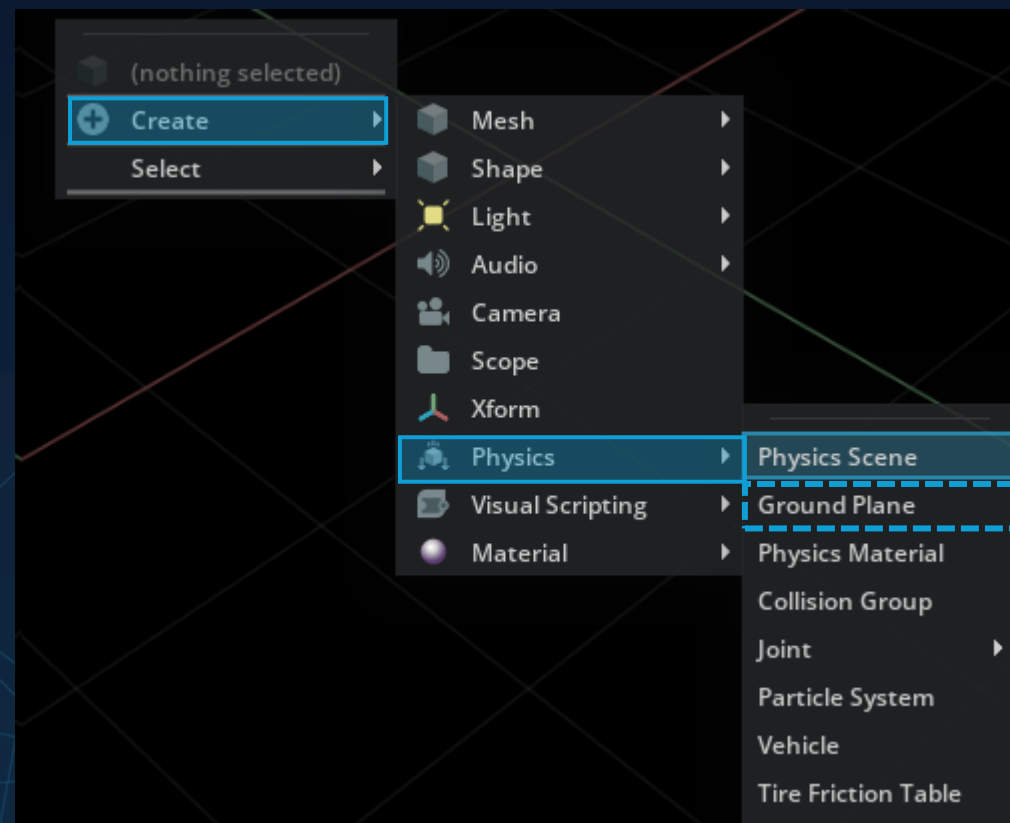
- Check for extensions:
 - Action Graph
 - URDF importer





How-to: Add Physics Scene

- Create a Physics Scene
- Create a ground plane



- ## TYPES OF ASSETS

-

©Hochschule Kempten – University of Applied Sciences

How-to: Use Asset Store



File Edit Create Window cuOpt Tools Utilities Layouts Help

RTX - Real-Time Perspective Stage Lights

1. Search
2. Drag & Drop
(to Stage or Viewport)

FPS: 118.36, Frame time: 8.45 ms
NVIDIA L40S: 1.6 GiB used, 37.1 GiB available
Process Memory: 5.4 GiB used, 50.7 GiB available
1920x1080

a

b

Isaac Sim Asset Store

Box

1

2

ALL 1193
+ ENVIRONMENTS 70
+ ISAACLAB 62
+ MATERIALS
+ PEOPLE 312
+ PROPS 456
+ ROBOTS 117
+ SAMPLES 113
+ SENSORS 63

003_cracker_box 003_cracker_box 004_sugar_box 004_sugar_box 008_pudding_box

009_gelati sm_whitec sm

File name: 003_cracker_box.usd
File size: 460 KB
File Path: Props/YCB/Axis_Aligned/003_cracker_box.usd

Stage Layer Render Settings

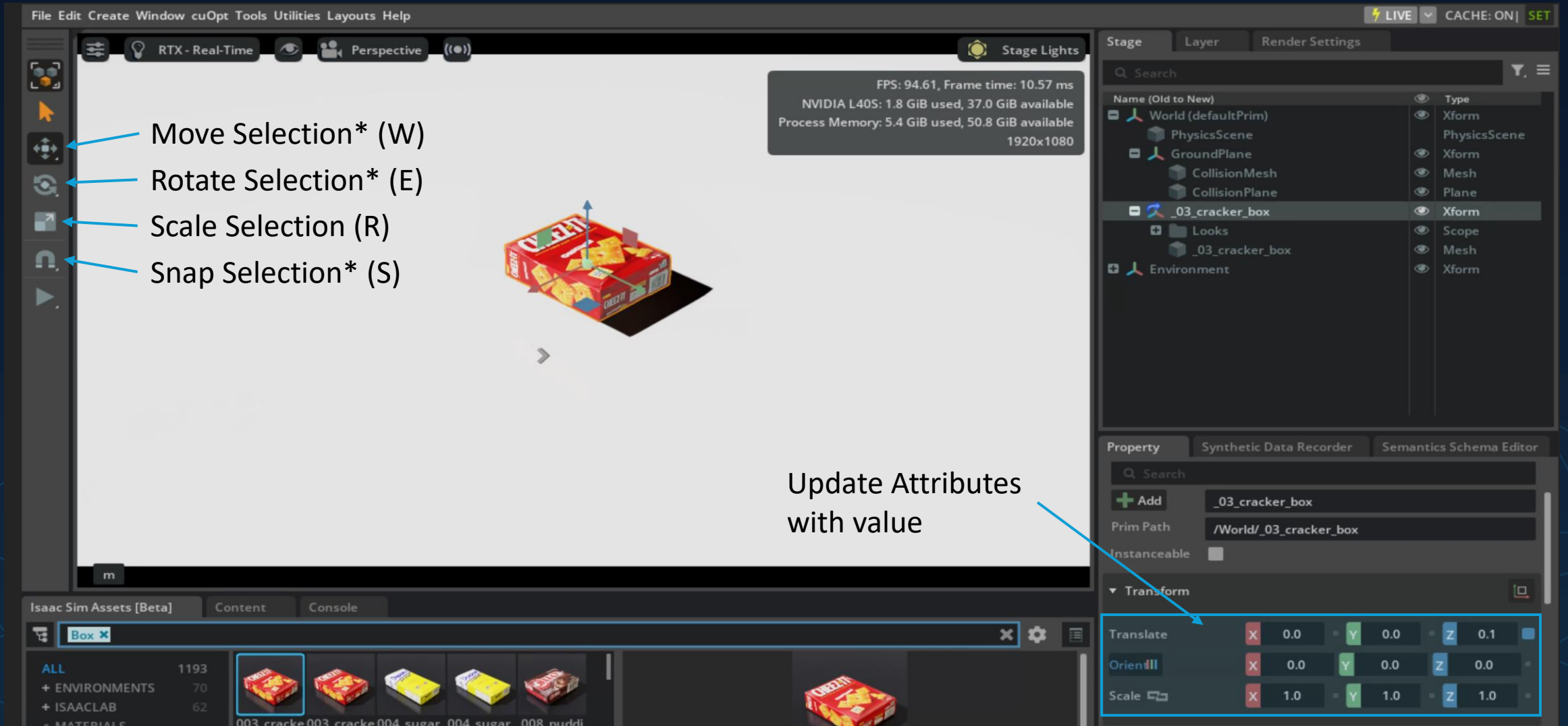
Search

Name (Old to New)	Type
World (defaultPrim)	Xform
PhysicsScene	PhysicsScene
GroundPlane	Xform
Environment	Xform

Property Synthetic Data Recorder Semantics Schema Editor

Search

How-to: Manipulation in Viewport



The screenshot displays the Isaac Sim interface with the following components:

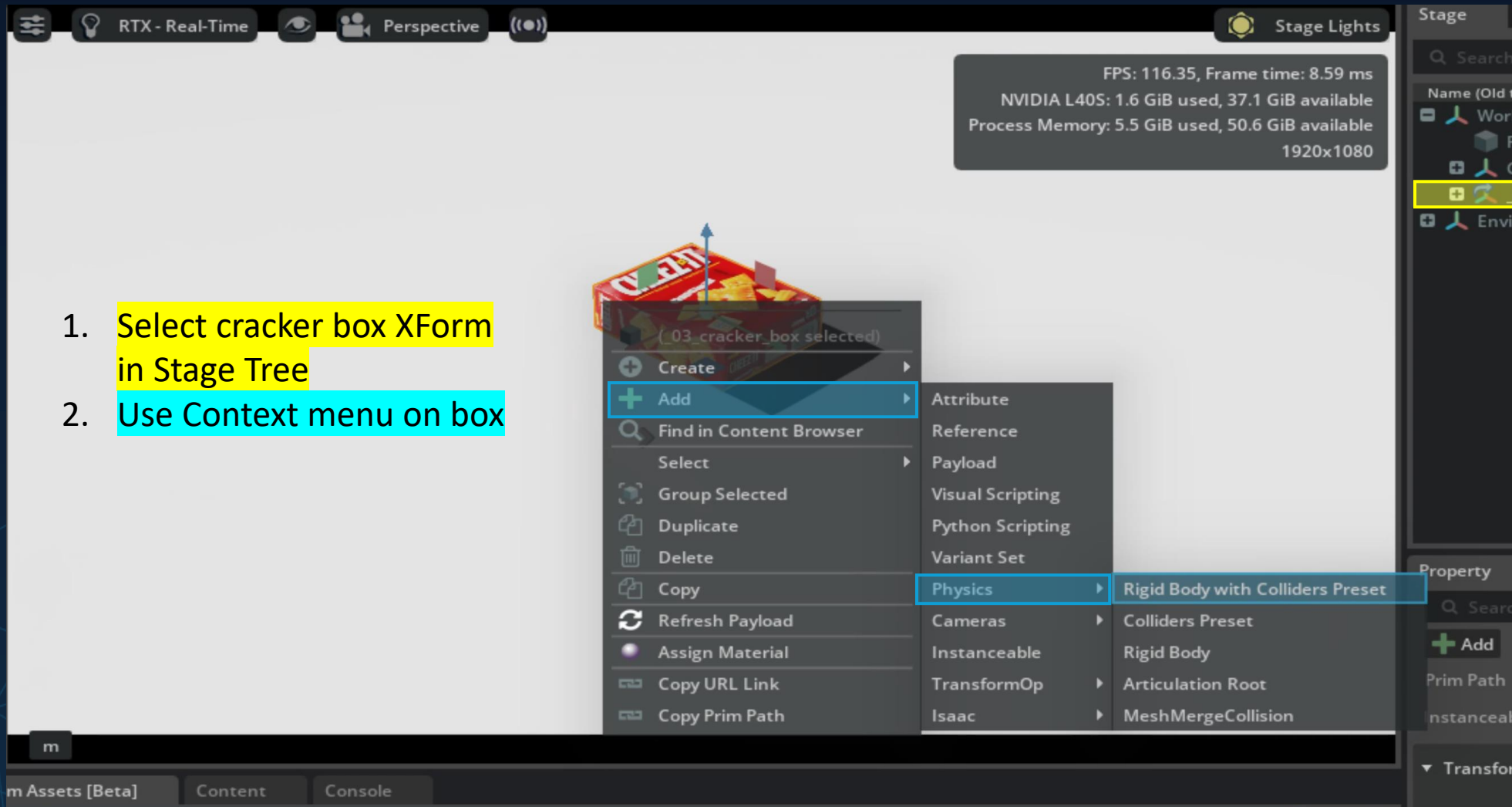
- Top Menu:** File, Edit, Create, Window, cuOpt, Tools, Utilities, Layouts, Help.
- Viewport:** Shows a 3D scene with a red cracker box. A status box in the top right corner displays performance metrics: FPS: 94.61, Frame time: 10.57 ms, NVIDIA L40S: 1.8 GiB used, 37.0 GiB available, Process Memory: 5.4 GiB used, 50.8 GiB available, 1920x1080.
- Left Toolbar:** Contains icons for Move Selection* (W), Rotate Selection* (E), Scale Selection (R), and Snap Selection* (S).
- Right Panel:** Includes a Stage panel with a search bar and a list of objects. The selected object is **_03_cracker_box** (Xform). Below it is the Property panel, which shows the object's path as **/World/_03_cracker_box** and its transform properties.
- Bottom Panel:** Shows the Isaac Sim Assets [Beta] section with a search bar and a list of assets. The selected asset is **Box**.

Annotations in the image include:

- Blue arrows pointing to the Move Selection* (W), Rotate Selection* (E), Scale Selection (R), and Snap Selection* (S) icons in the left toolbar.
- A blue arrow pointing to the **Translate** property in the Property panel, with the text "Update Attributes with value" next to it.

Property	X	Y	Z
Translate	0.0	0.0	0.1
Orientation	0.0	0.0	0.0
Scale	1.0	1.0	1.0

How-to: Add Rigid-body & Collider

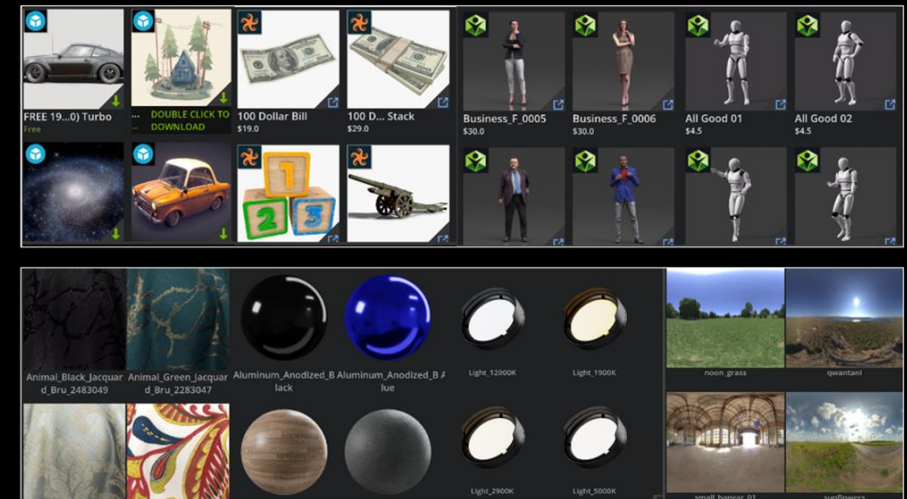


Asset Store

- Import a Box
 - Move above ground plane
 - Adjust the size
 - Add (Rigid Body and and Collider)
 - Press *Play* to start the Simulation

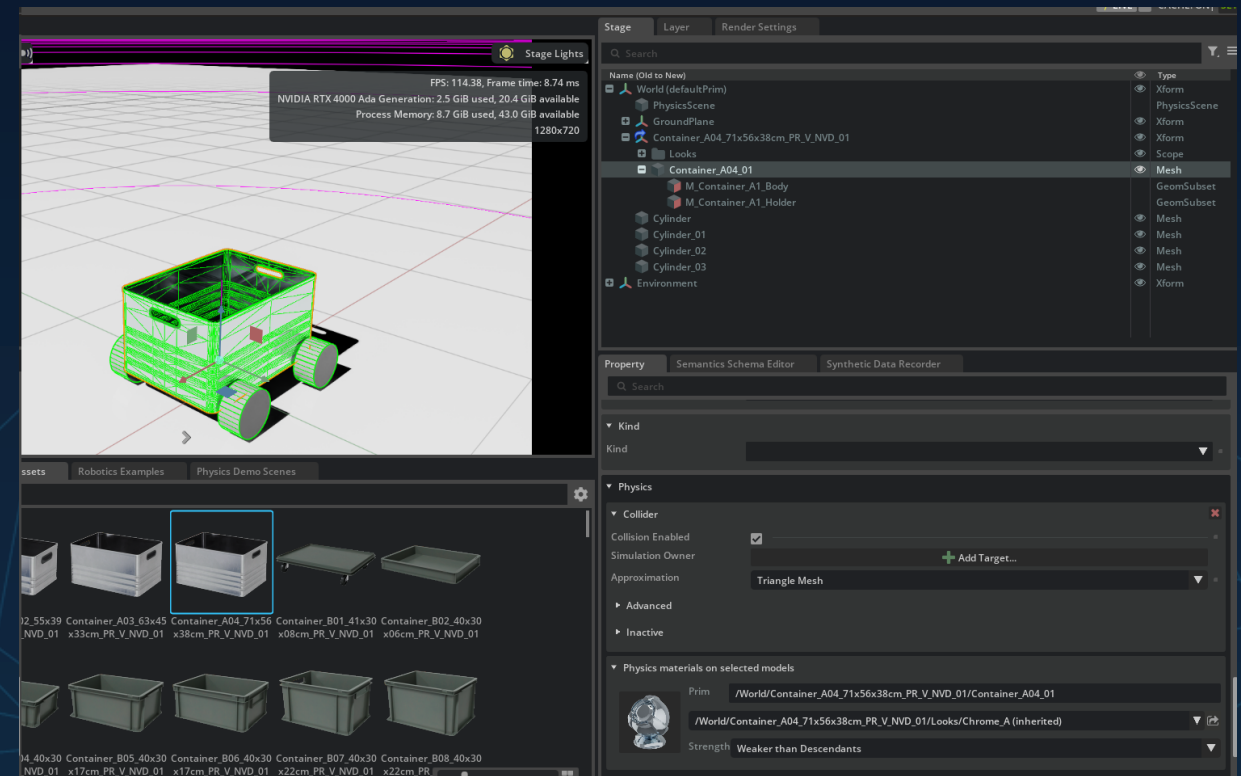
TYPES OF ASSETS

- 3D Models
- Motions/Animations
- Materials/Shaders
- Lights/Light Sets
- Environments/
Scenes/HDRIs
- Simulation-Ready
Assets
- Procedurally
Generated
Assets/Brushes
- Volumetric Effects e.g.
Smoke, fog

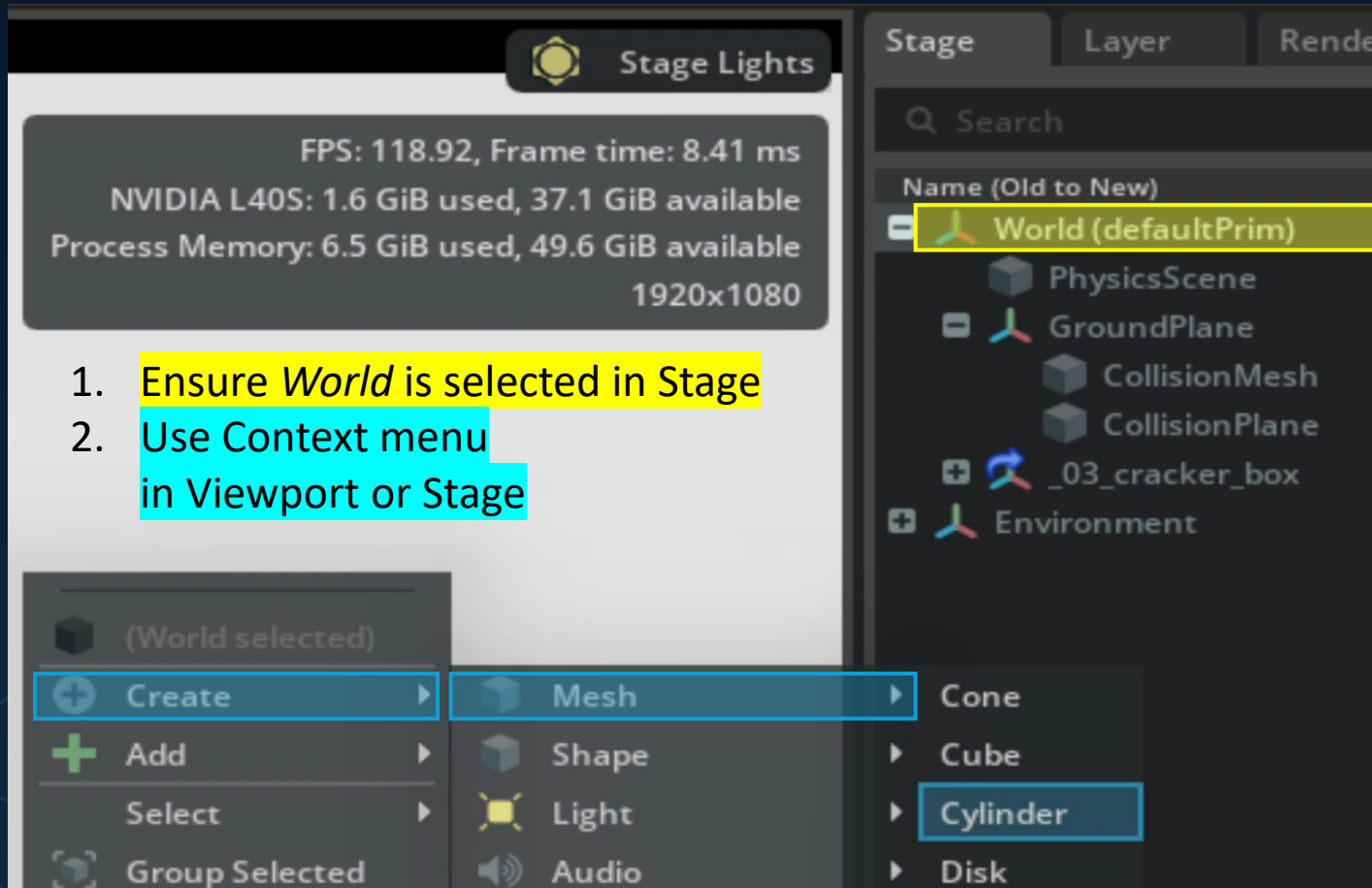


Assemble the first self-built Robot

- Stop the Simulation
- Create Cylinders
 - Move/ orient to the side
 - Create a revolute joint
 - Add Angular drive
 - Modify target velocity + stiffness
- Start the Simulation



How-to: Add Cylinder



1. Ensure *World* is selected in Stage
2. Use Context menu in Viewport or Stage

3. Adjust scaling and orientation to represent a wheel
4. Add *Rigid Body & Collider*
5. Use *Ctrl+C* and *Ctrl+V* to replicate the wheel
6. Move the wheels to reasonable positions around the box

How-to: Add Joint

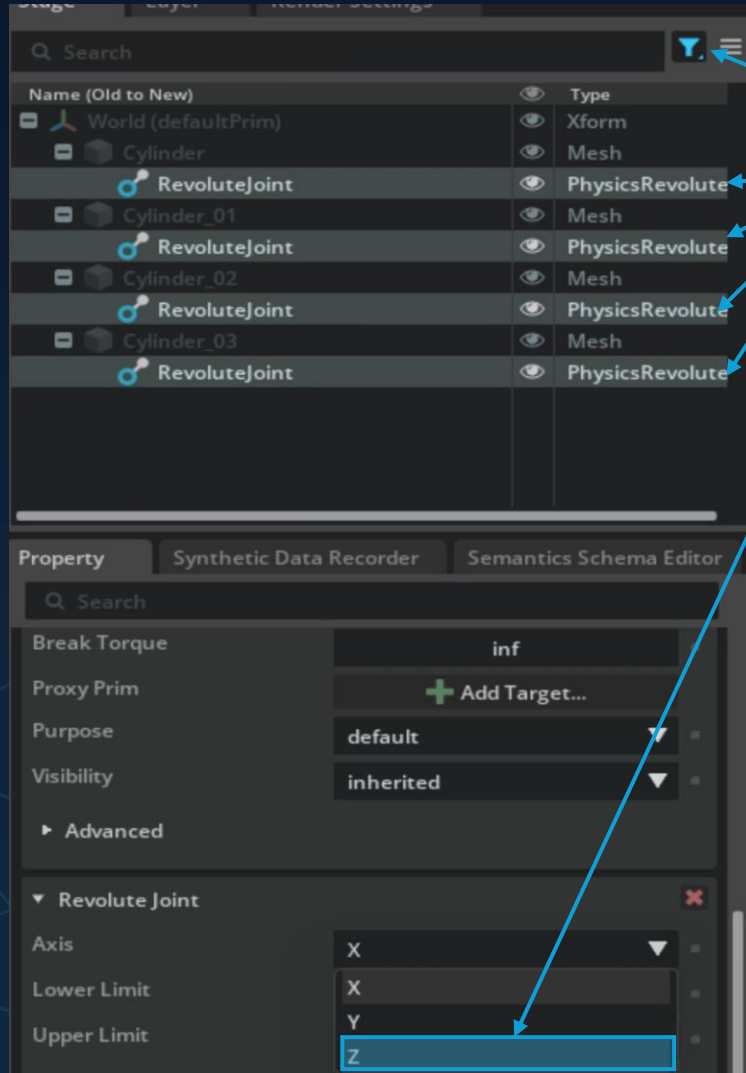


NVIDIA L40S: 1.6 GiB used, 37.1 GiB available
Process Memory: 6.7 GiB used, 49.4 GiB available
1920x1080

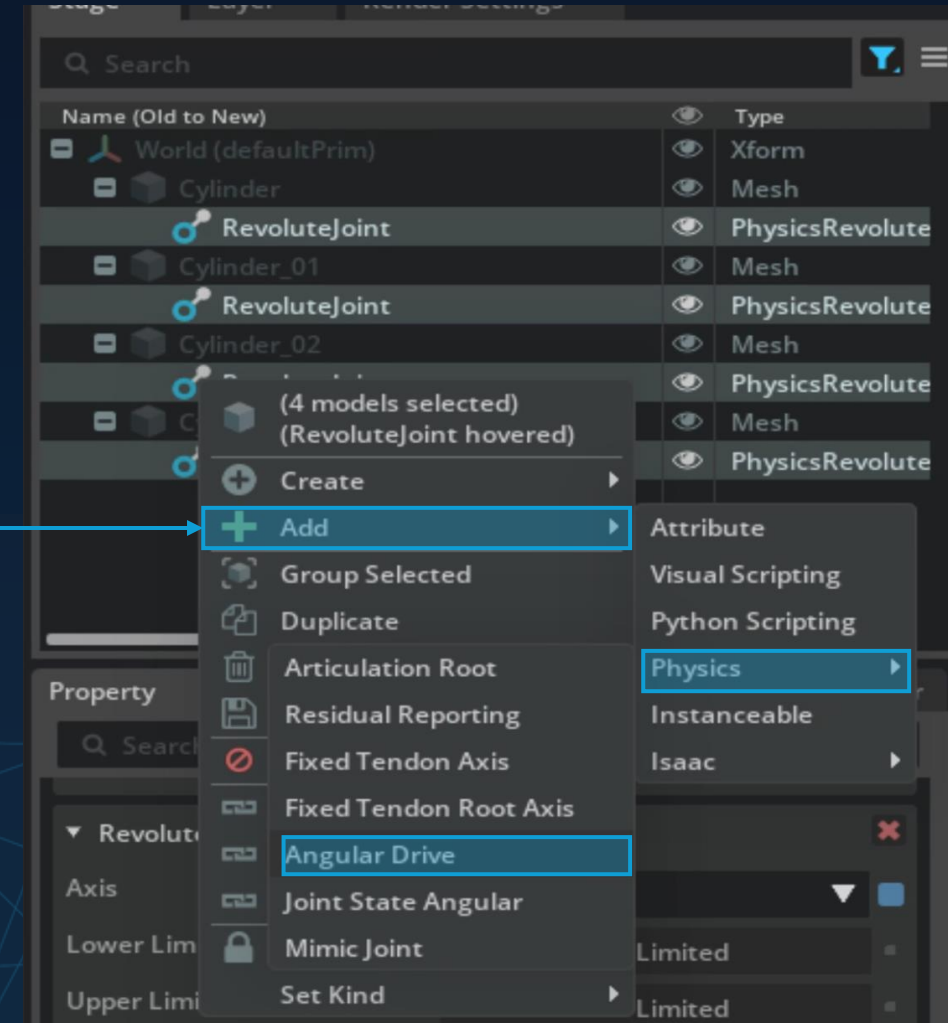
1. Holding *Ctrl*, select the box then the cylinder
2. Create joint using the context menu
3. Repeat for all *wheels*

The screenshot shows a 3D software interface with a car model. A red box labeled 'CHEEZ-IT' is on top of the car. A cylinder (wheel) is selected. A context menu is open, showing the 'Physics' menu with 'Joint' selected, and a sub-menu showing 'Revolute Joint'.

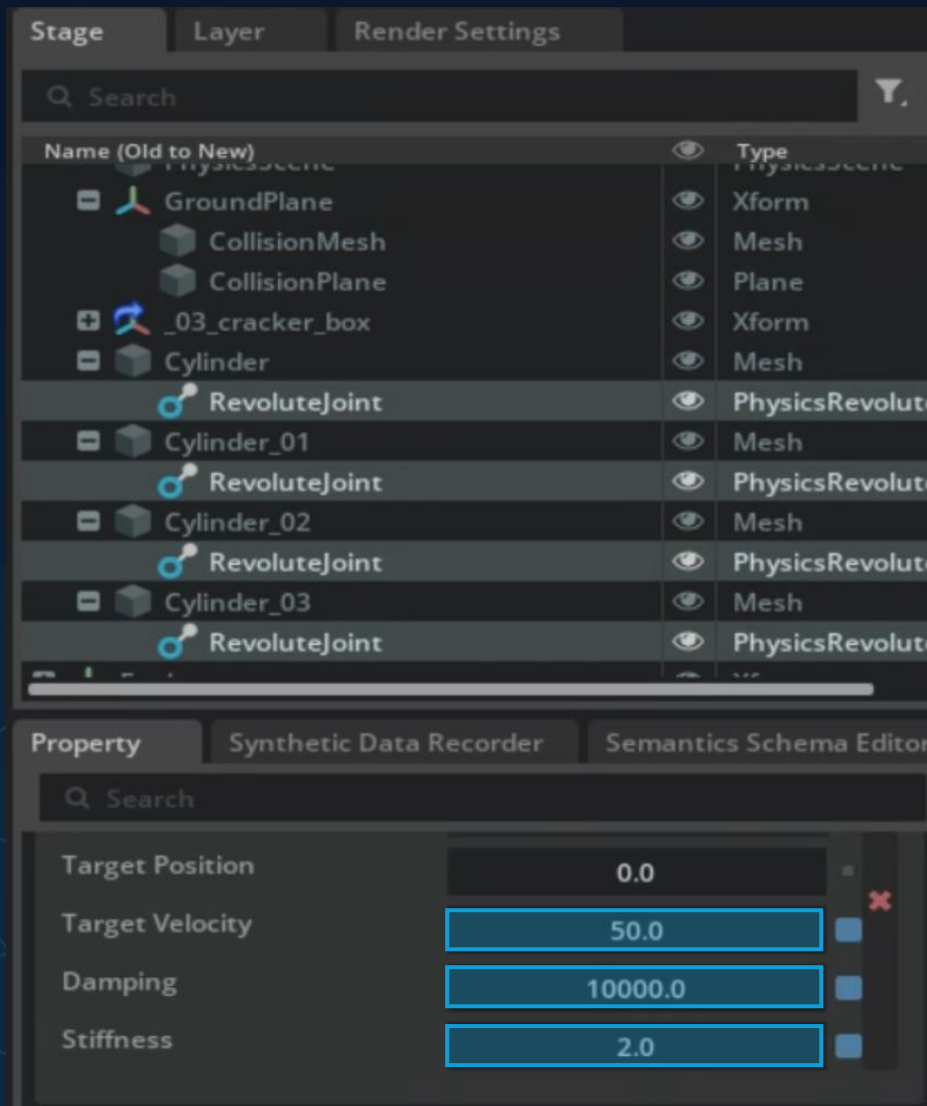
How-to: Adjust Joint and Add Drive



1. Filter for Joints
2. Select all joints
(via *Shift* or *Ctrl*)
3. Adjust joint rotation
4. Add Drives to joints
(via Context menu)



How-to: Adjust Drive

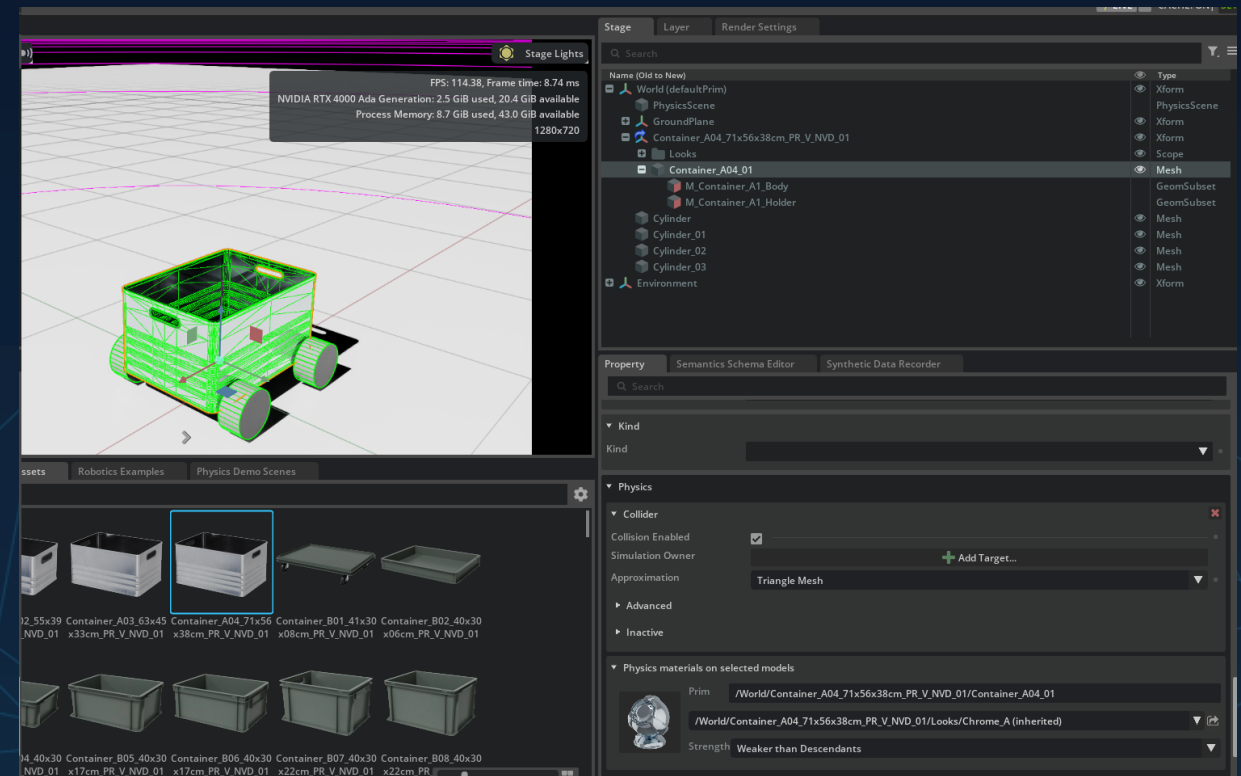


1. Set a Velocity(~20-100)
2. Iterate:
 1. Set Damping & Stiffness Values
 2. Restart Simulation for Testing*

*Not restarting the simulation leads to misleading results due to the forces that already applied before the update

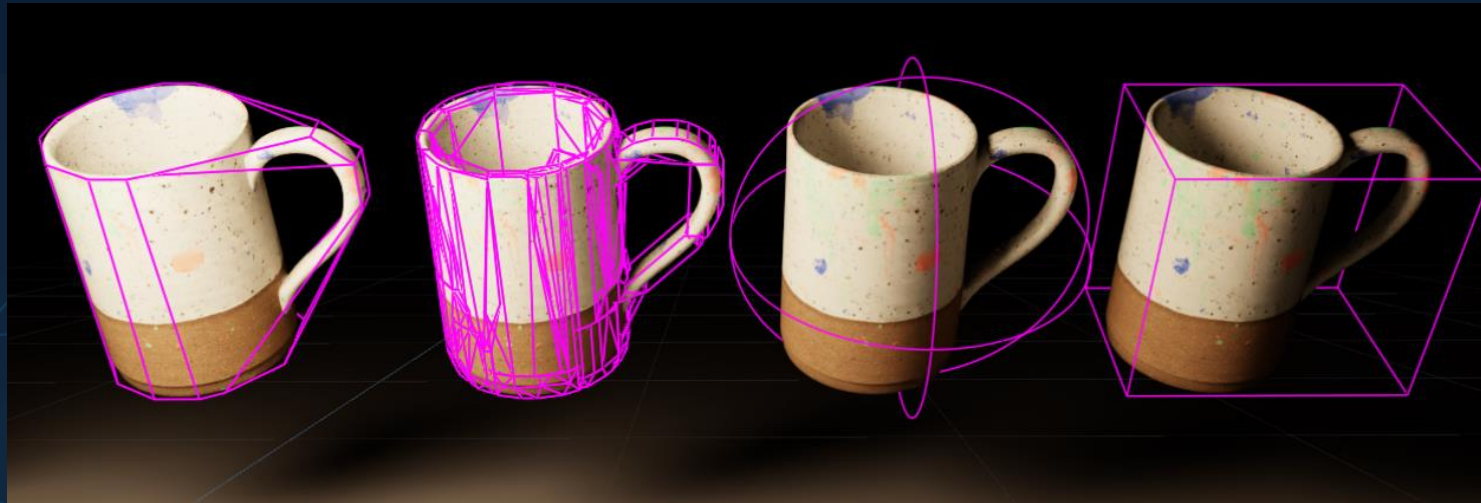
Assemble the first self-built Robot

- Stop the Simulation
- Create Cylinders
 - Move/ orient to the side
 - Create a revolute joint
 - Add Angular drive
 - Modify target velocity + stiffness
- Start the Simulation

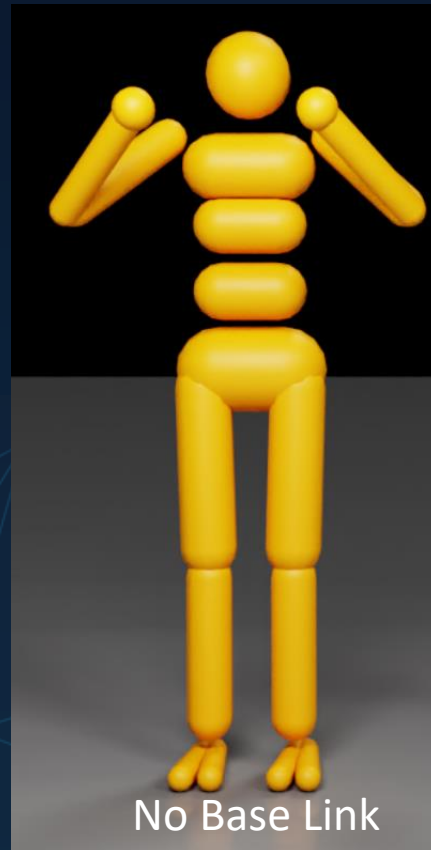


Rigid Bodies and Colliders

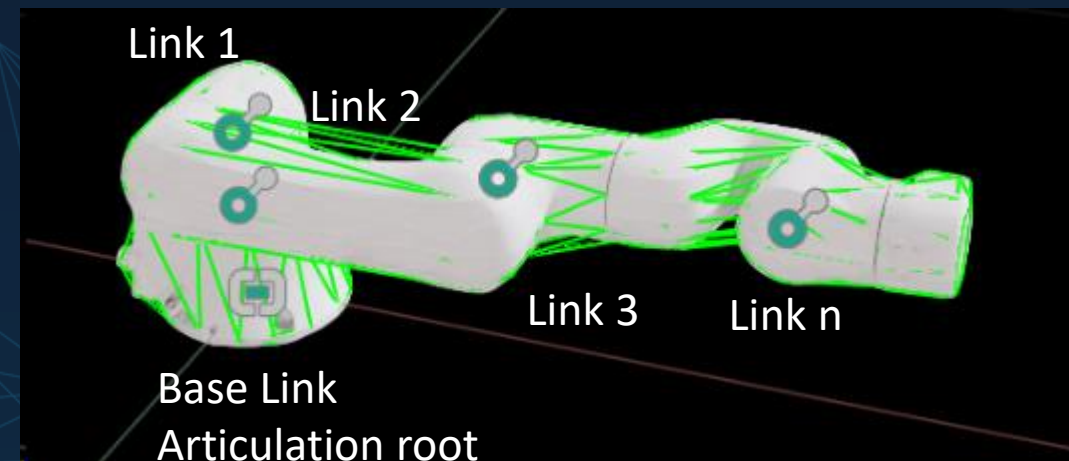
- Simpler approximation = faster Simulation



Robot assembly

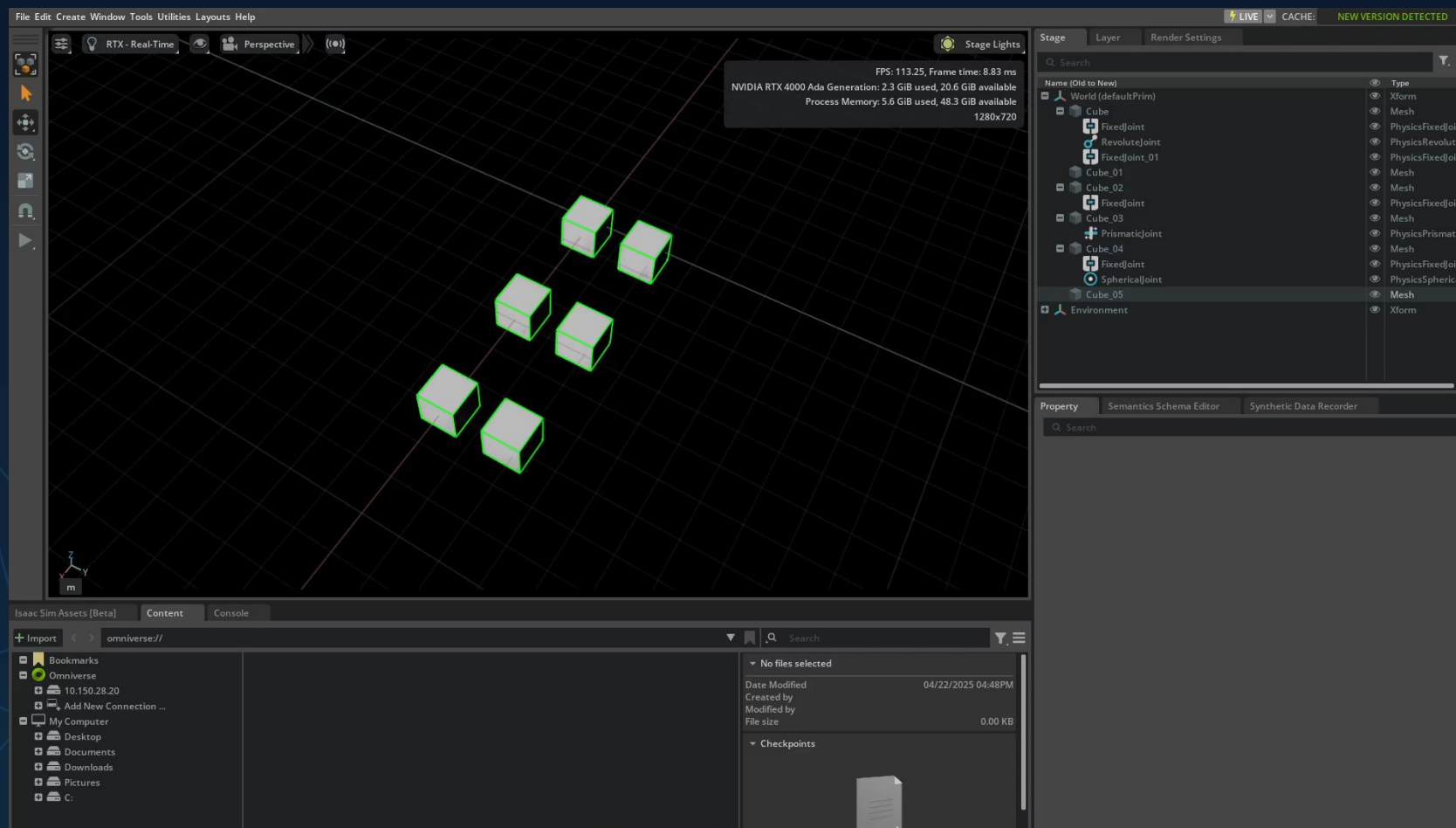


- Structure of mobile / static robot
 - Articulation chain
 - root link free / fixed



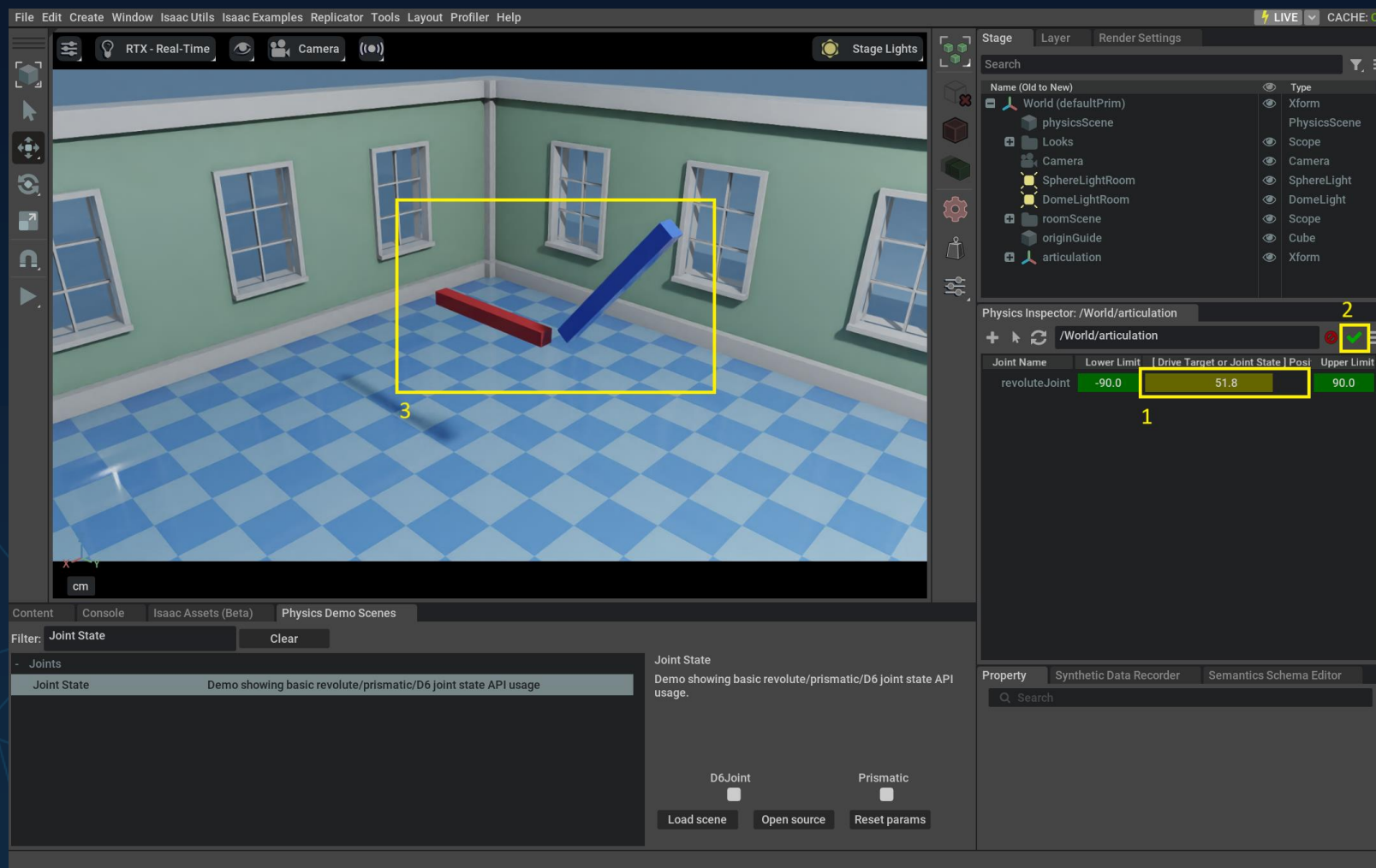


Joint types and properties

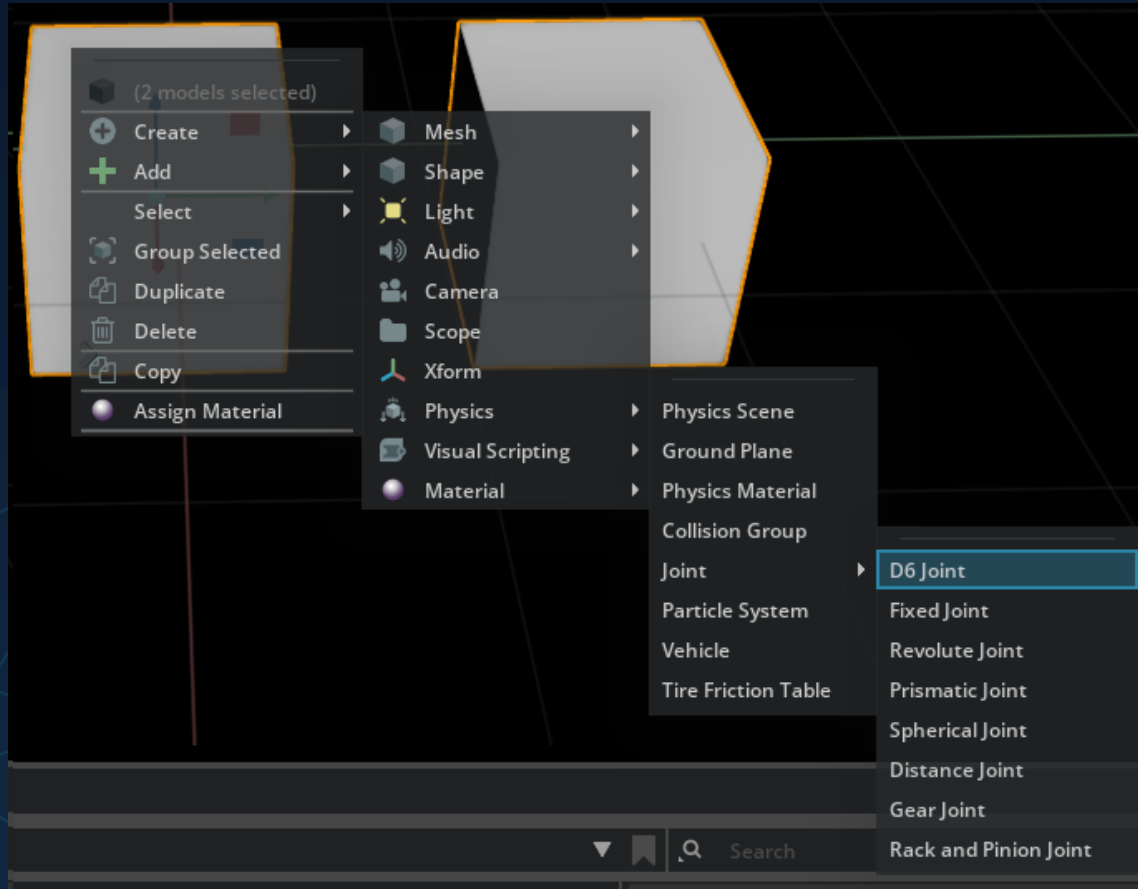




Joints



Joint types and properties



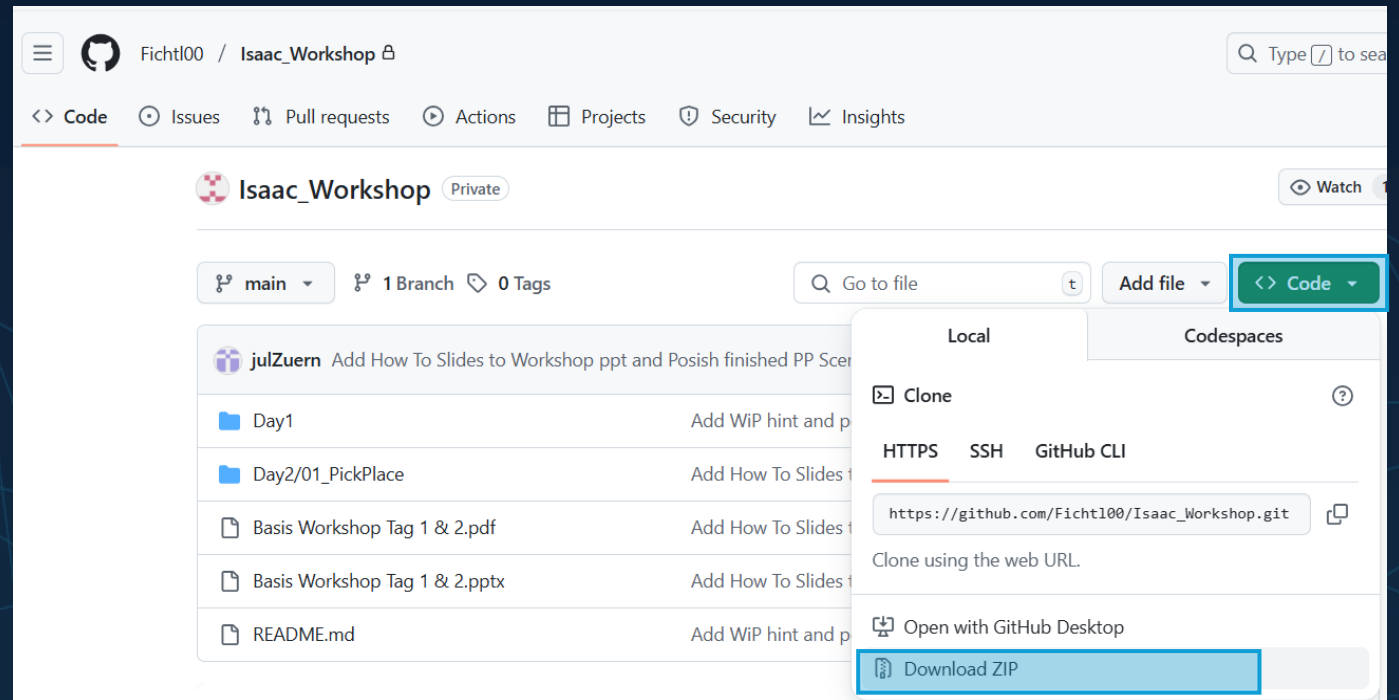
Joint drives are internally simulated with PD-Controllers

- *Stiffness* property $\leftrightarrow$ Proportional coefficient
- *Damping* property $\leftrightarrow$ Derivative coefficient

⚠ This can lead to unexpected behavior
at low simulation frequencies

How-to: Clone the Git Repository

- Go to the link* provided in advance
- Download the Workshop Contents or Clone the Repository
- (Unzip the Package)



Until now:

- We reviewed necessary Omniverse Basics and Resources
- Practical session:
 - Extensions installation
 - Scene Manipulation

10 min Break

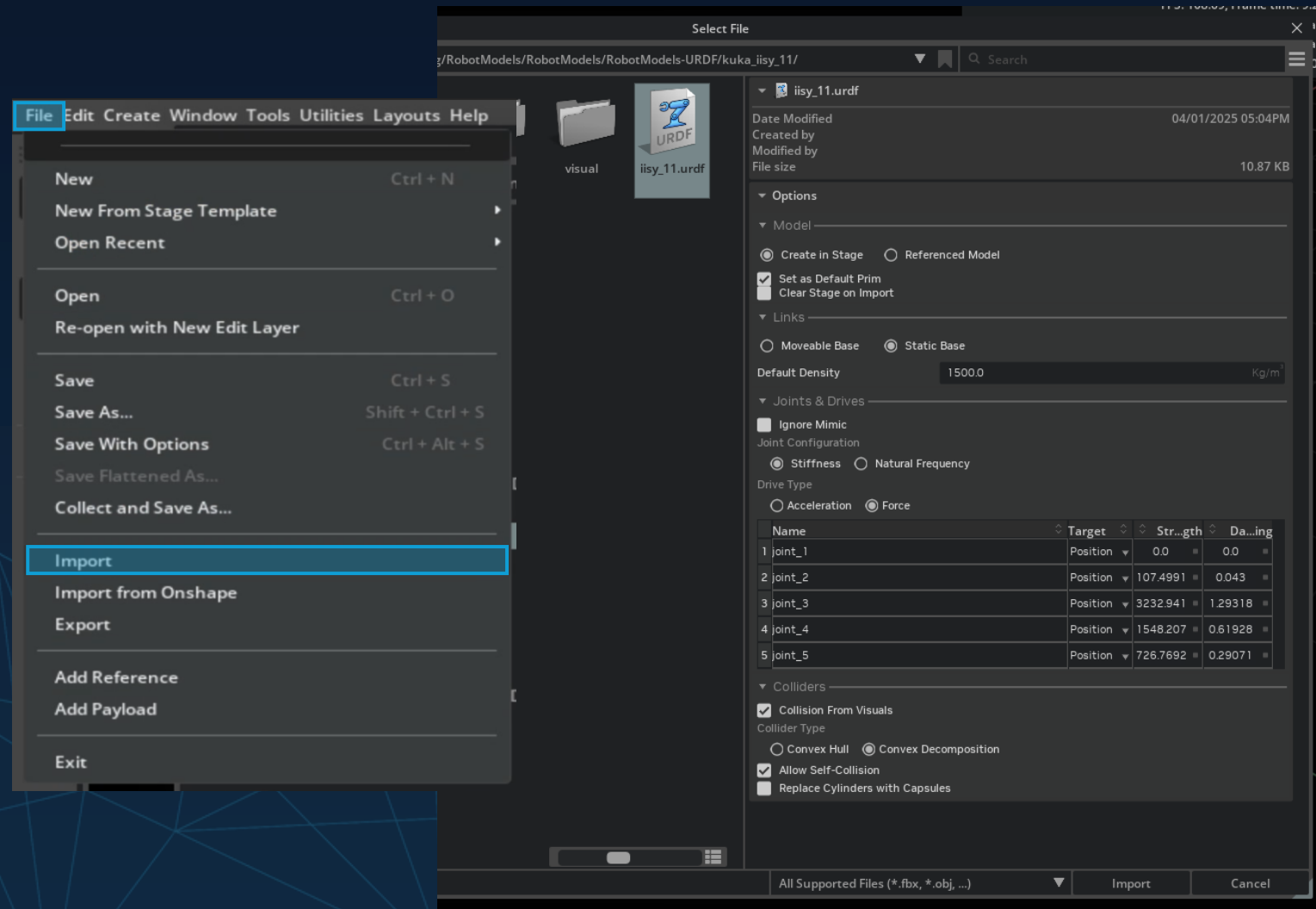
To be continued:

- Practical session:
 - Robot import + tuning



How-to: Starting with a real Robot

- Open *File*->*import*
- Navigate to a URDF File
- Modify the Options
 - Verify *Model* and *Links*
 - Adjust *Joints & Drives*
 - Verify *Colliders*
- Click *Import*





Robot import Tools

- LULA URDF Import Tool:
 - Import Robot URDF
 - Start scene -> look at joint drives
- Physics Inspector Tool:
 - Activate scene
 - Choose IISY to inspect-> control joint limits/ direction
- Gain tuner:
 - Adjust Stiffness & Damping and test the parameters
- Preparation of self and external collision shape:
 - Sphere approximation Collider tool for RMP

Physics Inspector: /lbr_iisy3_r760/Robotiq_2F_140_p...

+ ↩ ↺ /lbr_iisy3_r760/Robotiq_2F_140_physics_edit

Joint Name	Lower Limit	[Drive Target or Joir	Upper Limit
left_inner_knuckle_joint	Set Limit	0.0	Set Limit
right_inner_knuckle_joint	Set Limit	0.0	Set Limit

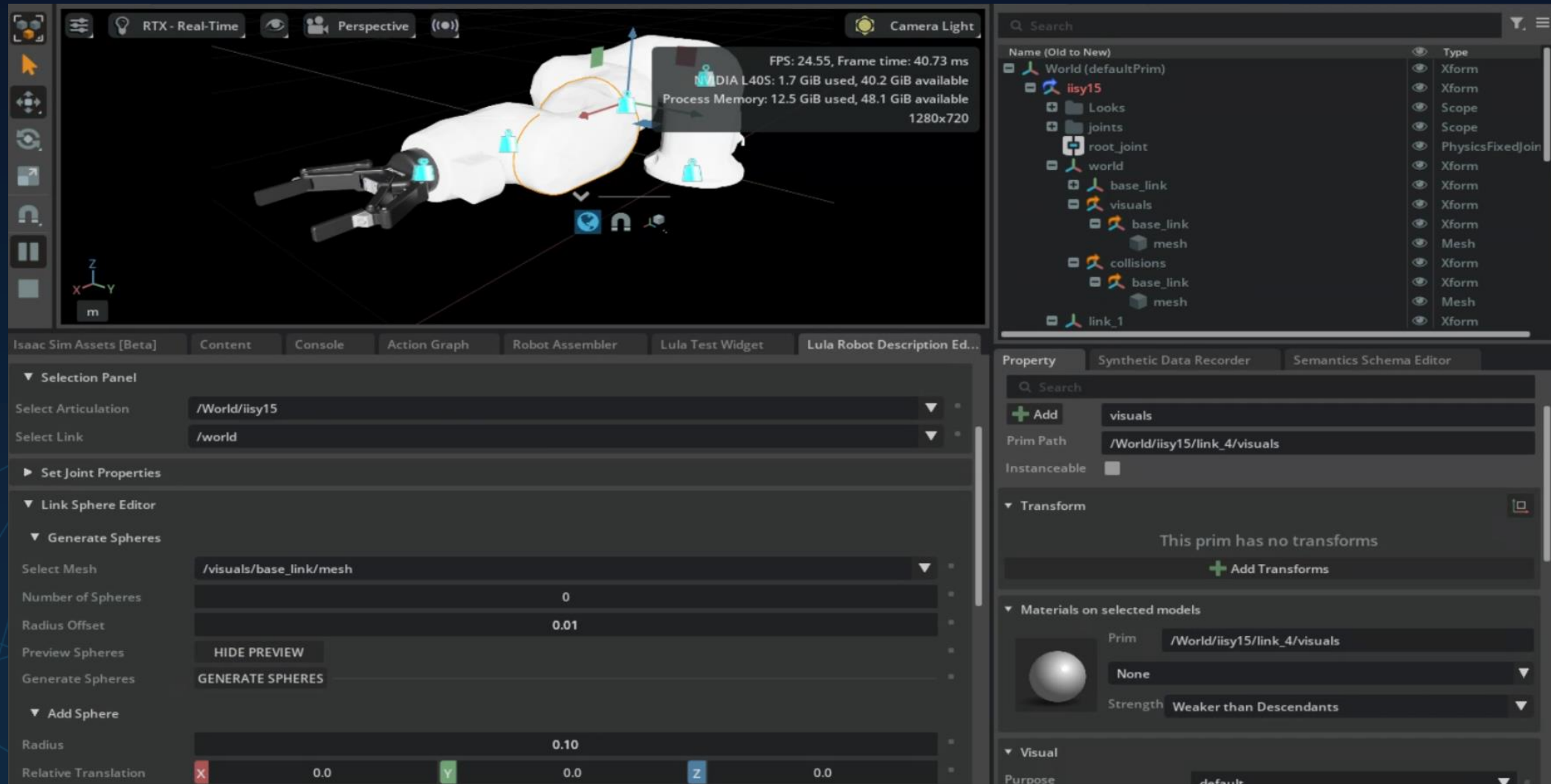
▼ Robotiq_2F_140_physics_edit

Joint Name	Lower Limit	[Drive Target or Joir	Upper Limit
joint_1	-185.0	-93.5	185.0
joint_2	-230.0	-77.0	50.0
joint_3	-150.0	69.6	150.0
joint_4	-175.0	16.2	175.0
joint_5	-110.0	64.6	110.0
joint_6	-220.0	-220.0	220.0
FixedJoint	Fixed Joint		
finger_joint	0.0	0.0	45.0
left_outer_finger_joint	0.0	0.0	180.0
left_inner_finger_joint	Set Limit	-42.7	Set Limit
left_inner_finger_pad_joint	-45.0	-45.0	45.0
right_outer_knuckle_joint	Mimic Joint		
right_outer_finger_joint	0.0	0.0	180.0
right_inner_finger_joint	Set Limit	-45.0	Set Limit
right_inner_finger_pad_joint	-45.0	0.0	45.0

Robot Description Editor

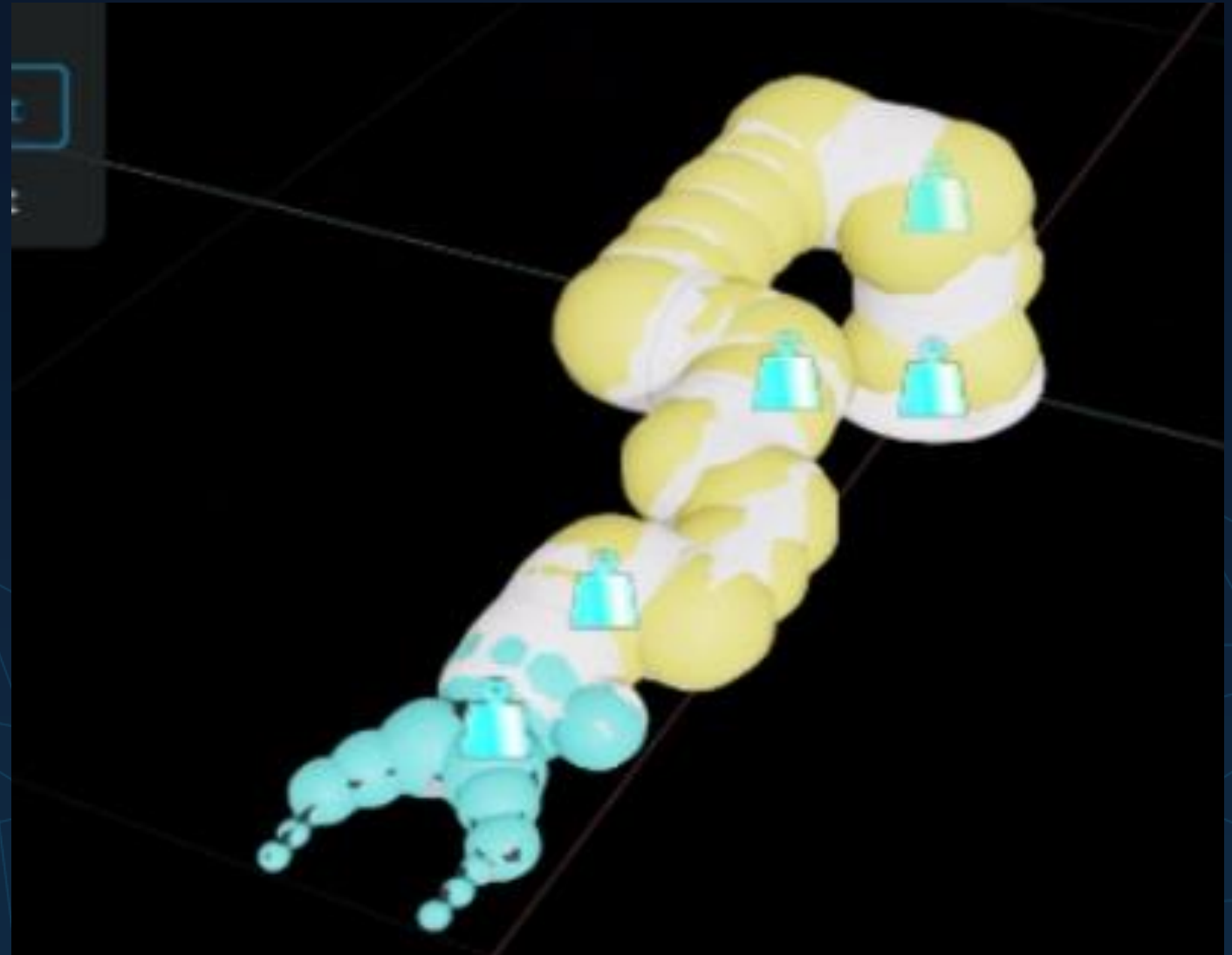


IPI
Institut für Produktion
und Informatik





Sphere Approximation Result



Lula Test Widget

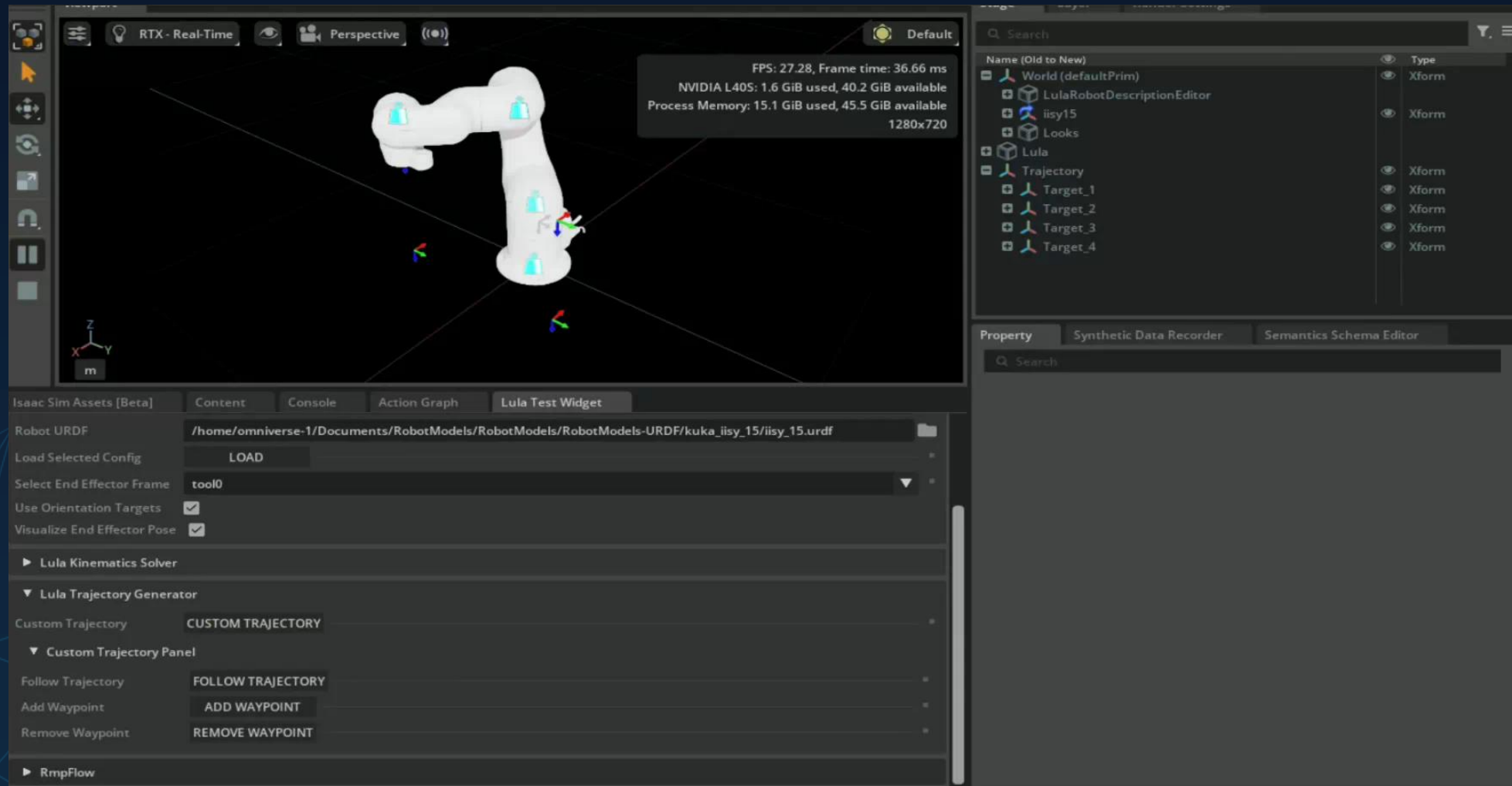


The screenshot displays the Lula Test Widget interface within a 3D simulation environment. The main viewport shows a white, multi-jointed robot model (Lula) in a perspective view. A status box in the top right corner of the viewport provides performance metrics: FPS: 27.04, Frame time: 36.98 ms, NVIDIA L40S: 1.7 GiB used, 40.1 GiB available, Process Memory: 15.1 GiB used, 45.6 GiB available, and a resolution of 1280x720.

The interface is divided into several panels:

- Selection Panel:** Contains fields for 'Select Articulation' (set to '/World/iisy15/root_joint'), 'Robot Description YAML' (set to '/home/omniverse-1/Documents/Workshop/iisy15.yaml'), 'Robot URDF' (set to '/home/omniverse-1/Documents/RobotModels/RobotModels-URDF/kuka_iisy_15/iisy_15.urdf'), 'Load Selected Config' (with a 'LOAD' button), 'Select End Effector Frame' (set to 'tool0'), 'Use Orientation Targets' (checked), and 'Visualize End Effector Pose' (checked).
- Lula Kinematics Solver:** A section for solving kinematics.
- Lula Trajectory Generator:** Includes a 'Custom Trajectory' field set to 'CUSTOM TRAJECTORY' and a 'Custom Trajectory Panel' with a 'Follow Trajectory' field set to 'FOLLOW TRAJECTORY'.
- Property Panel:** Located on the right, it shows a list of objects in the scene, including 'World (defaultPrim)', 'LulaRobotDescriptionEditor', 'iisy15', 'Looks', 'Lula', 'Trajectory', and four 'Target' objects. Below this, it displays 'Transform' properties (Translate, Rotate, Scale) and 'Materials on selected models'.

Lula Test Widget



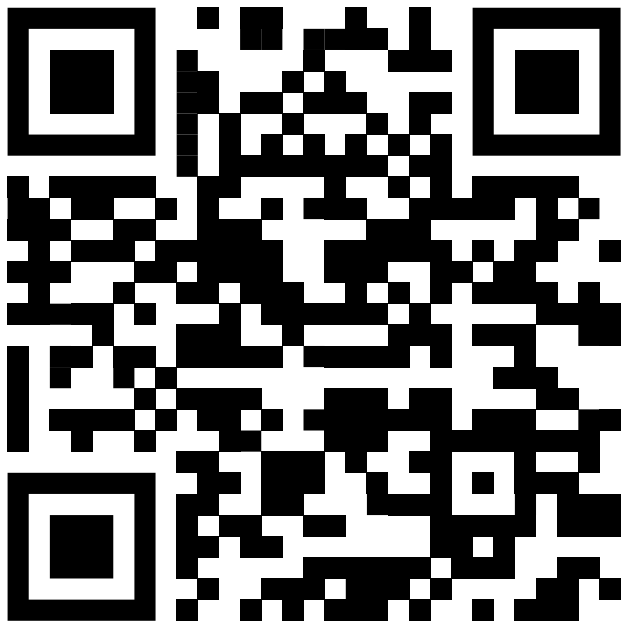
Sensors

- Adding a Wrist Camera to the Robot
 - Tune the Camera

End of Day 1

Please provide some Feedback

Verbal or via:



For further questions mail us:
fabian.fichtl@hs-kempten.de
julian.zuern@hs-kempten.de

Omniverse Base Workshop 1

KUKA X IPI

- 2x 4h Course with examples to
 - Learn the Basics in Omniverse, USD & Isaac Sim
 - Enable you to do Robotic model preparation & Simulation
 - Introduction to IsaacLab

Review of Day 1

- We reviewed necessary Omniverse Basics and Resources
- Practical session:
 - Extensions installation
 - Scene Manipulation
 - Robot import + tuning

Agenda Tag 2:

- Debugging Tools:
 - Console, VS Code, UI
- Scene Architecture
 - Build your Scenario: Environment, Robot, Gripper, Pick & Place
- Motion Planning & Motion Learning
- Materials
- Movie Capture Tool
- Information Channels | Further Courses

Interacting with USD (in Omniverse scene)

- UI Main Capabilities

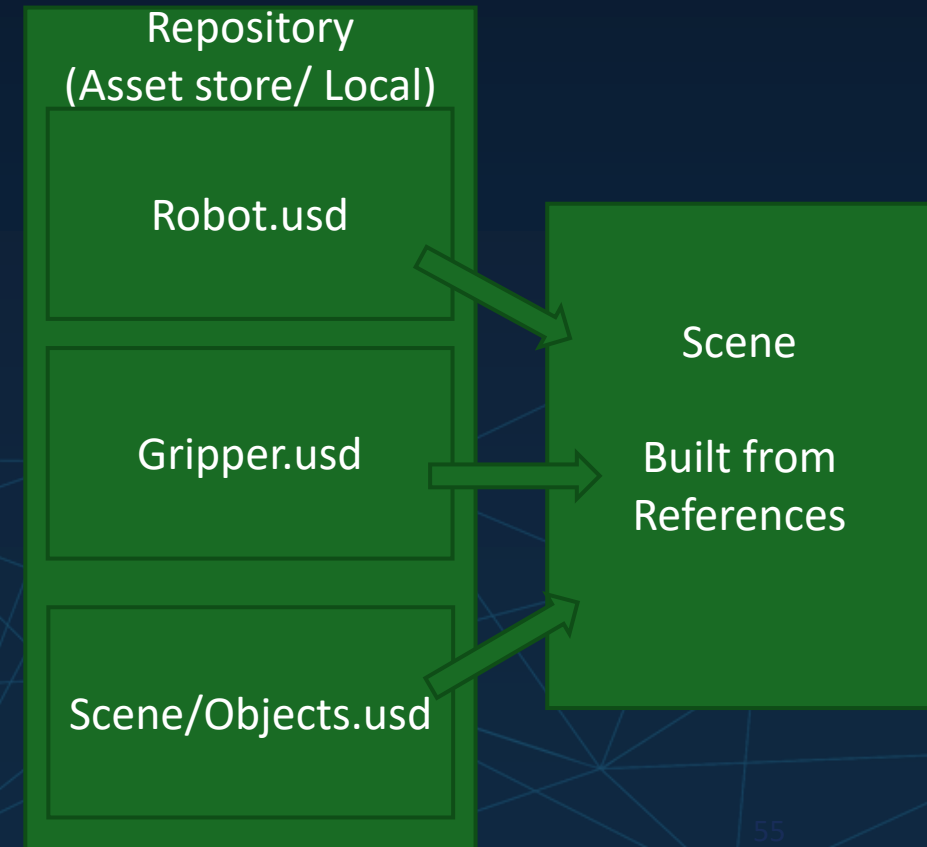
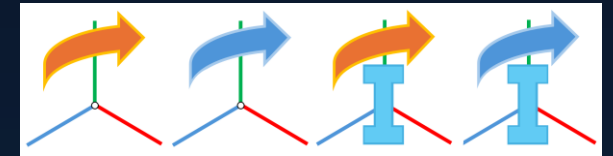
- Viewport:
 - Scene
- Property Window:
 - Prim* Properties
- Omnigraph:
 - Process Logic
 - ROS Controller
- Various extensions

- API Main Capabilities

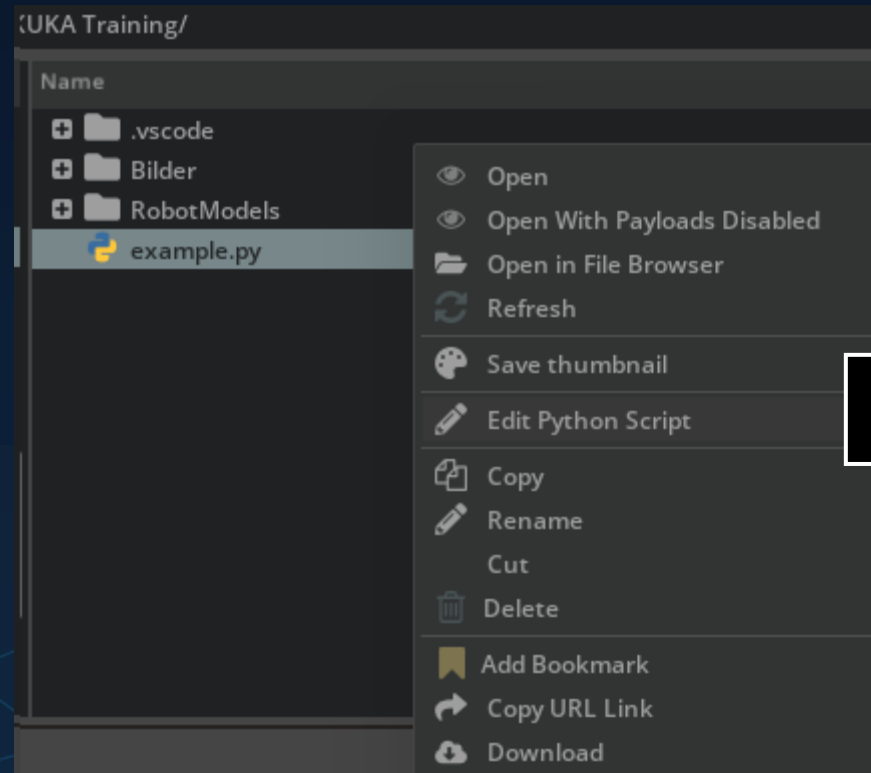
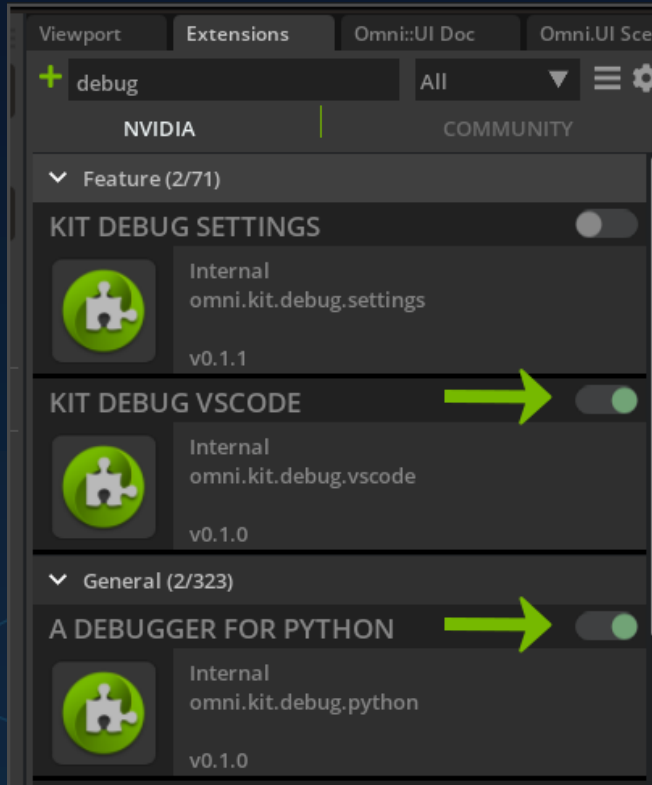
- Python interface
- Robot Controllers
- URDF import
- Create the scene for IsaacLab
- Configure the policy
- Start training

Scene Assembly

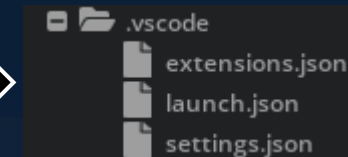
- Payload: blue ↔ Reference: orange
 - The user can choose to not load a payload
 - References always get loaded
 - I = Instancing active
- How to build a reference scene architecture
 - To test different robots/ grippers
 - Test Script of Robot Assembly



Debugging Tools



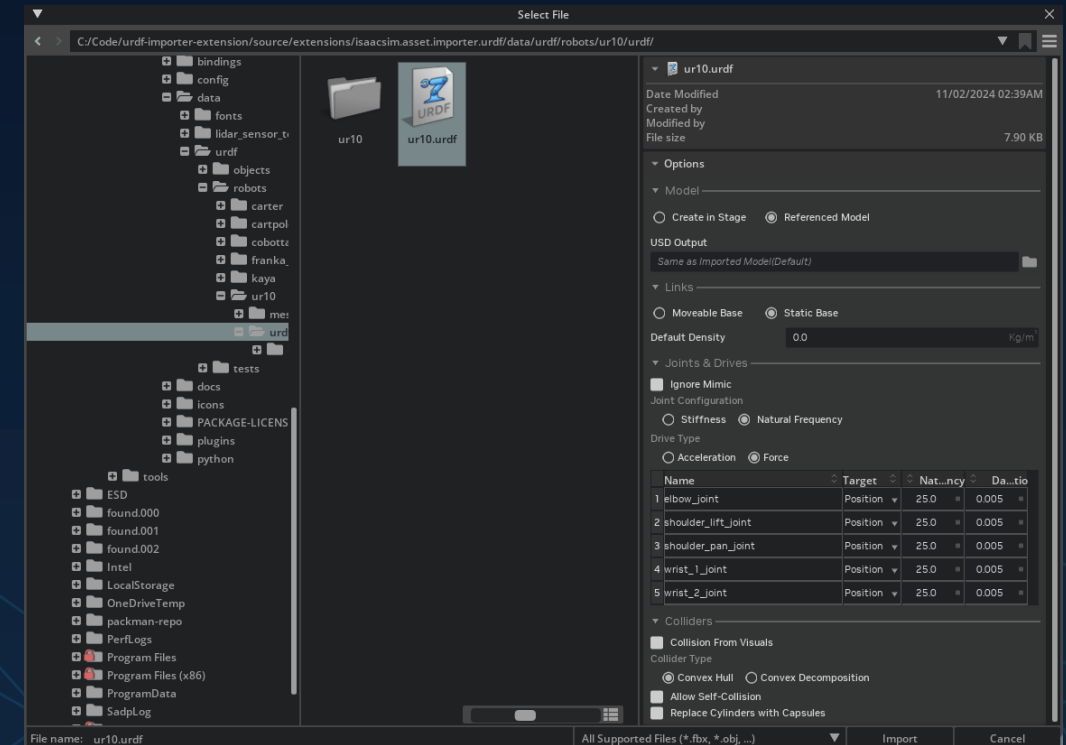
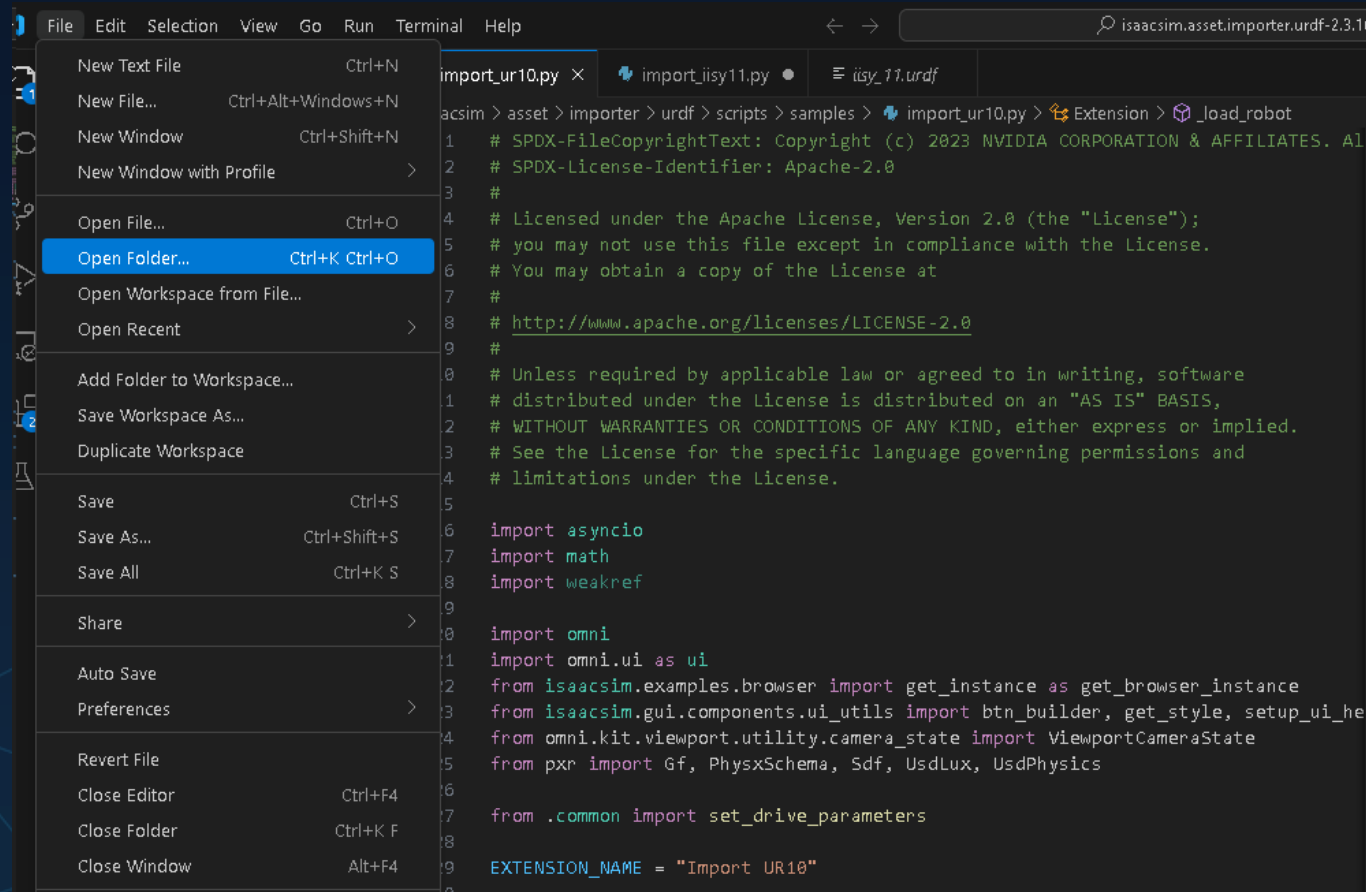
creates



https://docs.omniverse.nvidia.com/extensions/latest/ext_vs-code-link.html

**If you open .py in OV, the VS Code Link dependencies are created automatically
-> Enables hot reload of code**

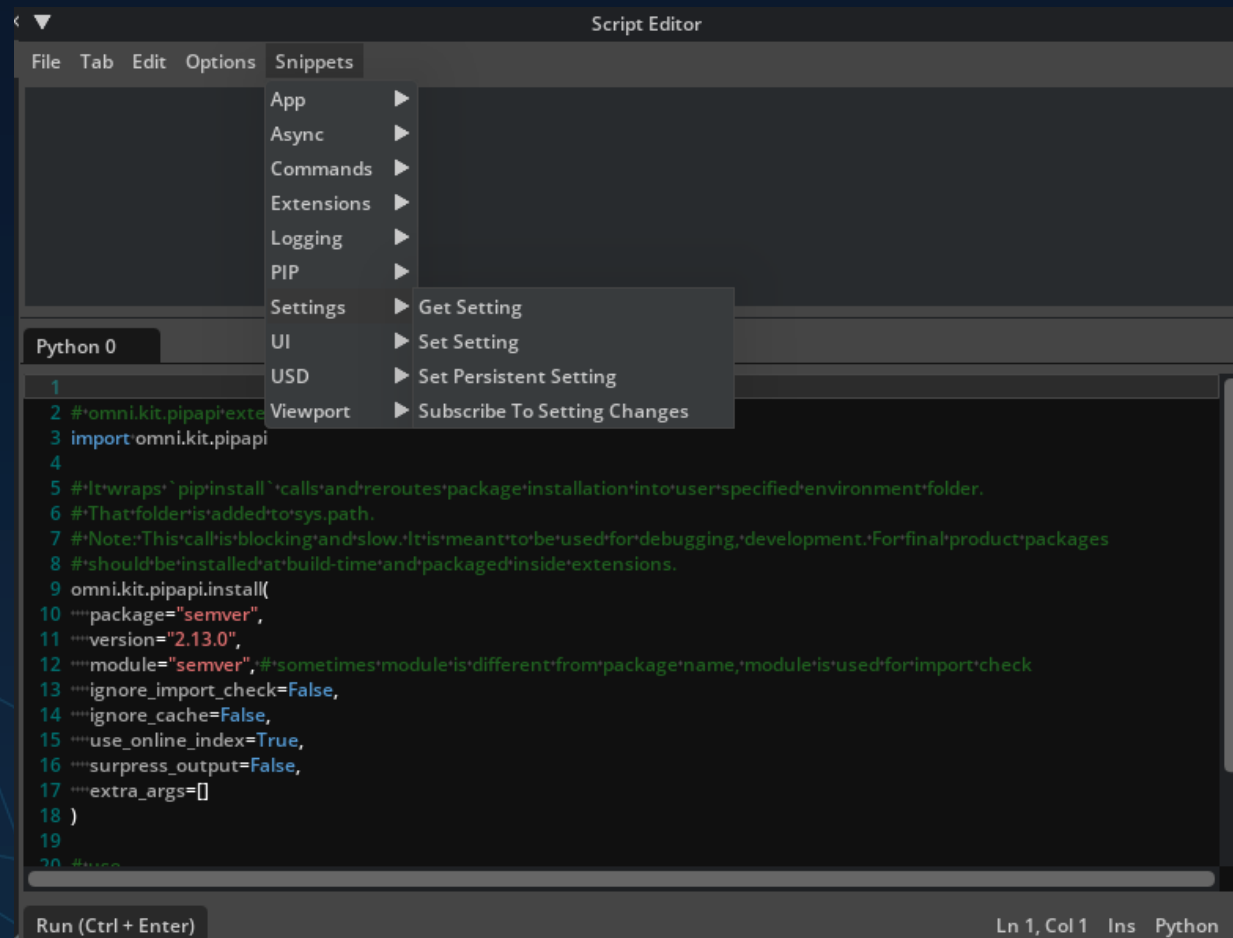
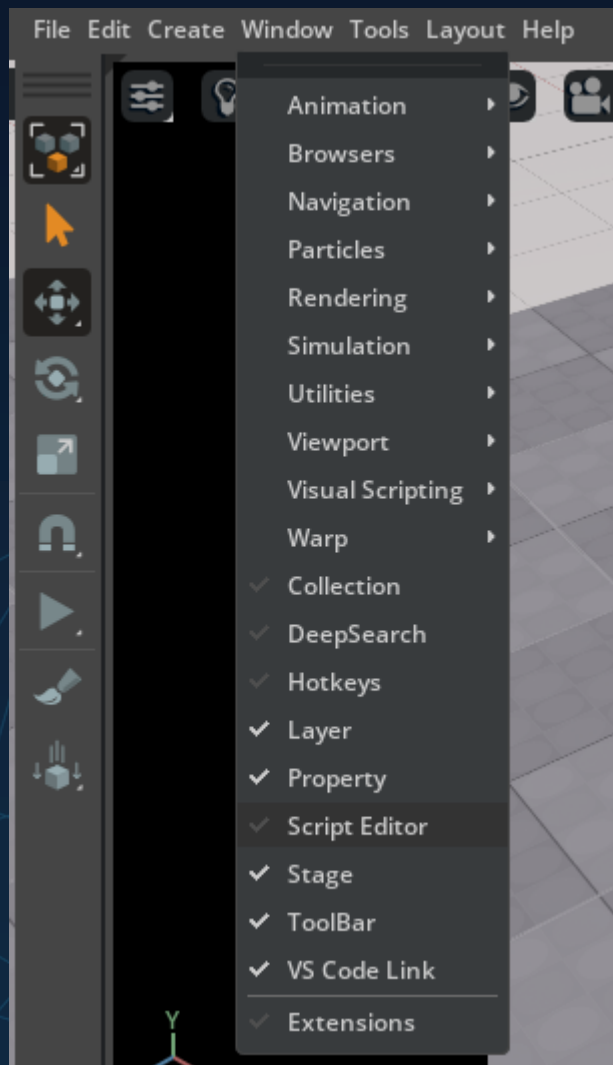
Debugging Tools



Helpful to reuse example code or look into USDA file



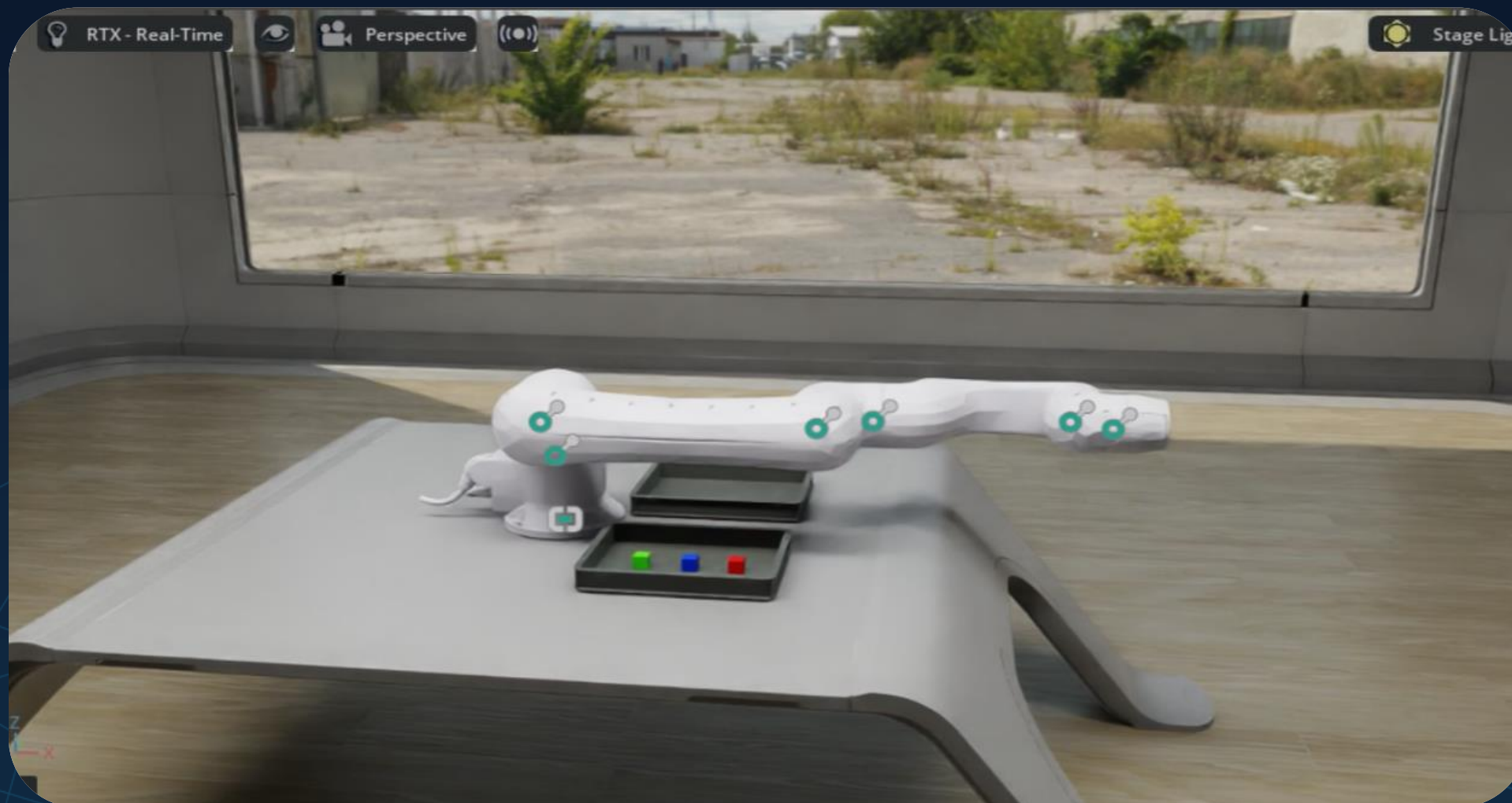
Debugging Tools



Script Editor



Assemble Your Scene





How-to: Assemble a gripper to your robot

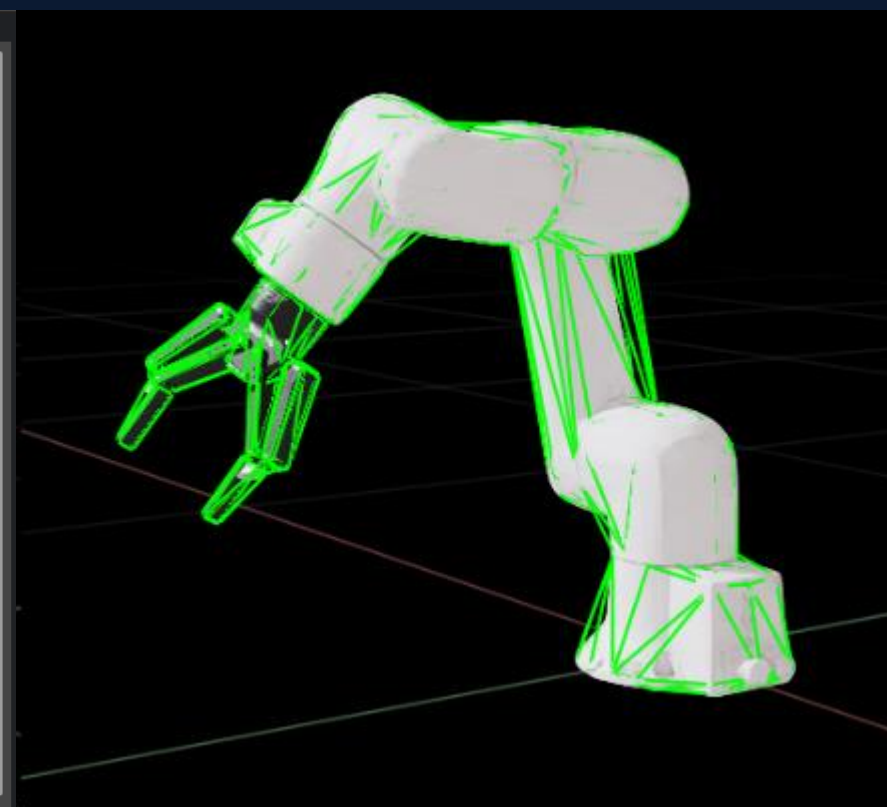
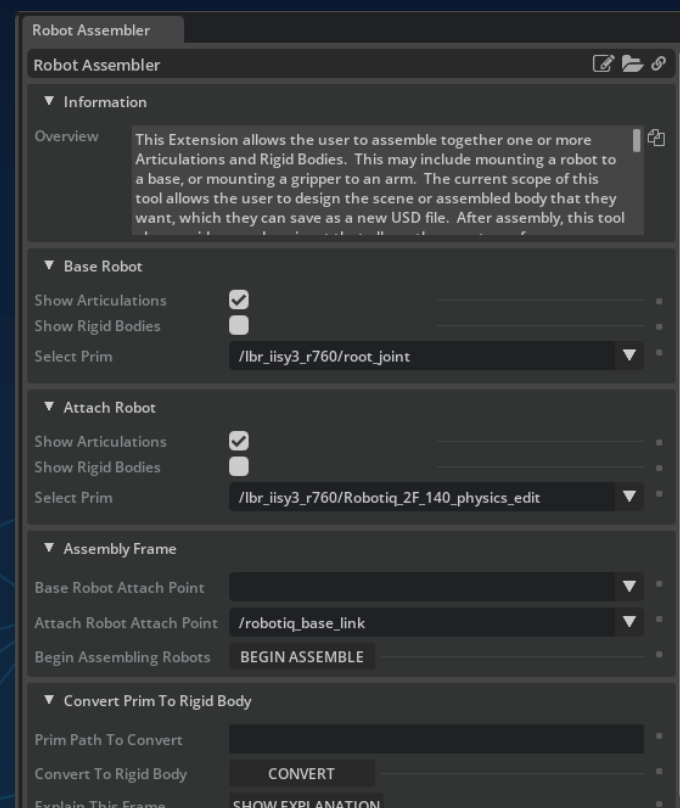
1. UI:

The Tool joins two articulation trees to one.

Guides you how

2. Python API

Code is created automatically. This can be reused later, automatically create a stage + assembly



How-to: Run Simple Scripts

```
import omni.usd
import omni.kit.commands from pxr
import Gf
# Get current stage
stage = omni.usd.get_context().get_stage()
# Create cube with specified parameters
result, prim_path = omni.kit.commands.execute(
    "CreateMeshPrim",
    prim_type="Cube",
    prim_path="/World/Cube",
    object_origin=Gf.Vec3f(0, 0, 0),
    half_scale=50.0,
    u_verts_scale=1,
    v_verts_scale=1,
    w_verts_scale=1 )
```

- Open the Script Editor
- Open the file *Day2/01_SceneAssembly/spawn_cube.py*.
- Analyze and run the Script
- Try to create another prim for example a cone:

```
# Create cone with specified parameters
result, prim_path = omni.kit.commands.execute(
    "CreateMeshPrim",
    prim_type="Cone",
    prim_path="/World/Cone",
    object_origin=Gf.Vec3f(0, 0, 0),
    half_scale=50.0, # 1m Height/Diameter (dep. on def.)
    u_verts_scale=1,
    v_verts_scale=1,
    w_verts_scale=1 )
```

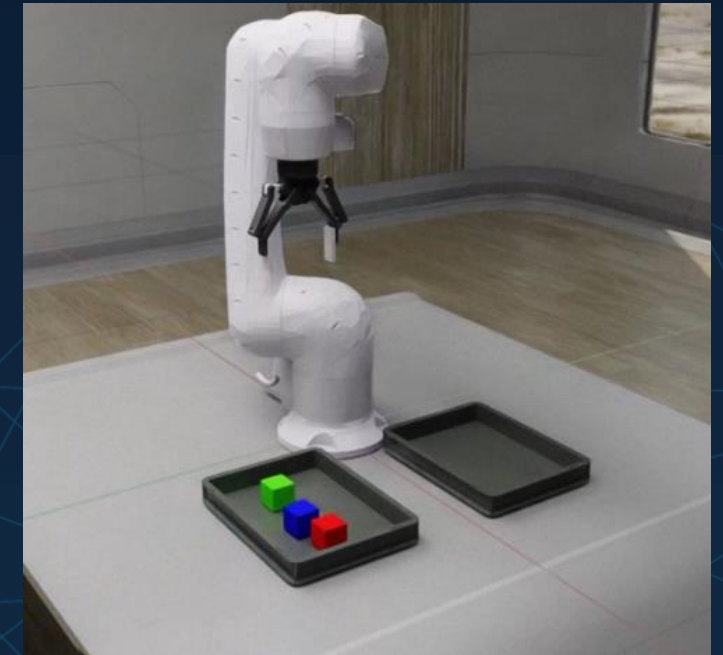
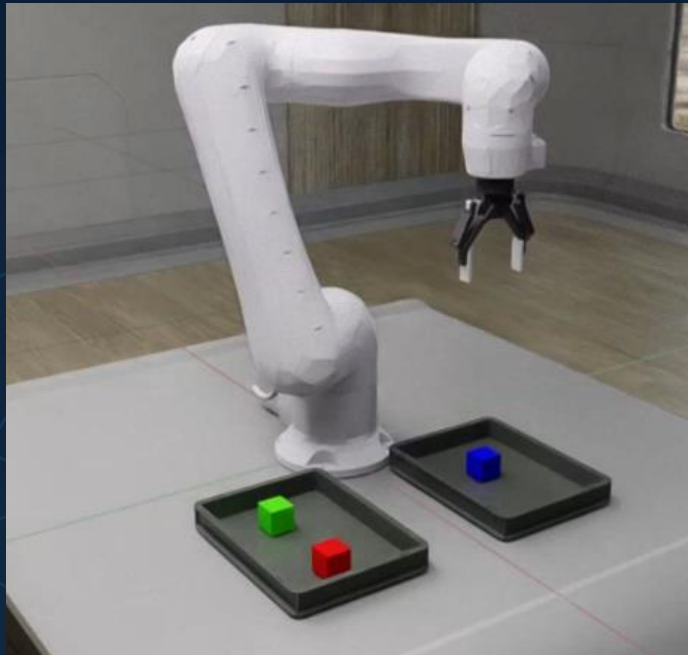

How-to: Run Assembly Scripts

- Create a new empty USD stage
- In the Script Editor
 - Open the file *Day2/01_SceneAssembly/assemble_scene.py*.
 - Analyze the Script and contained References
 - Run it
- Next, inspect and run the *assemble_robot.py*

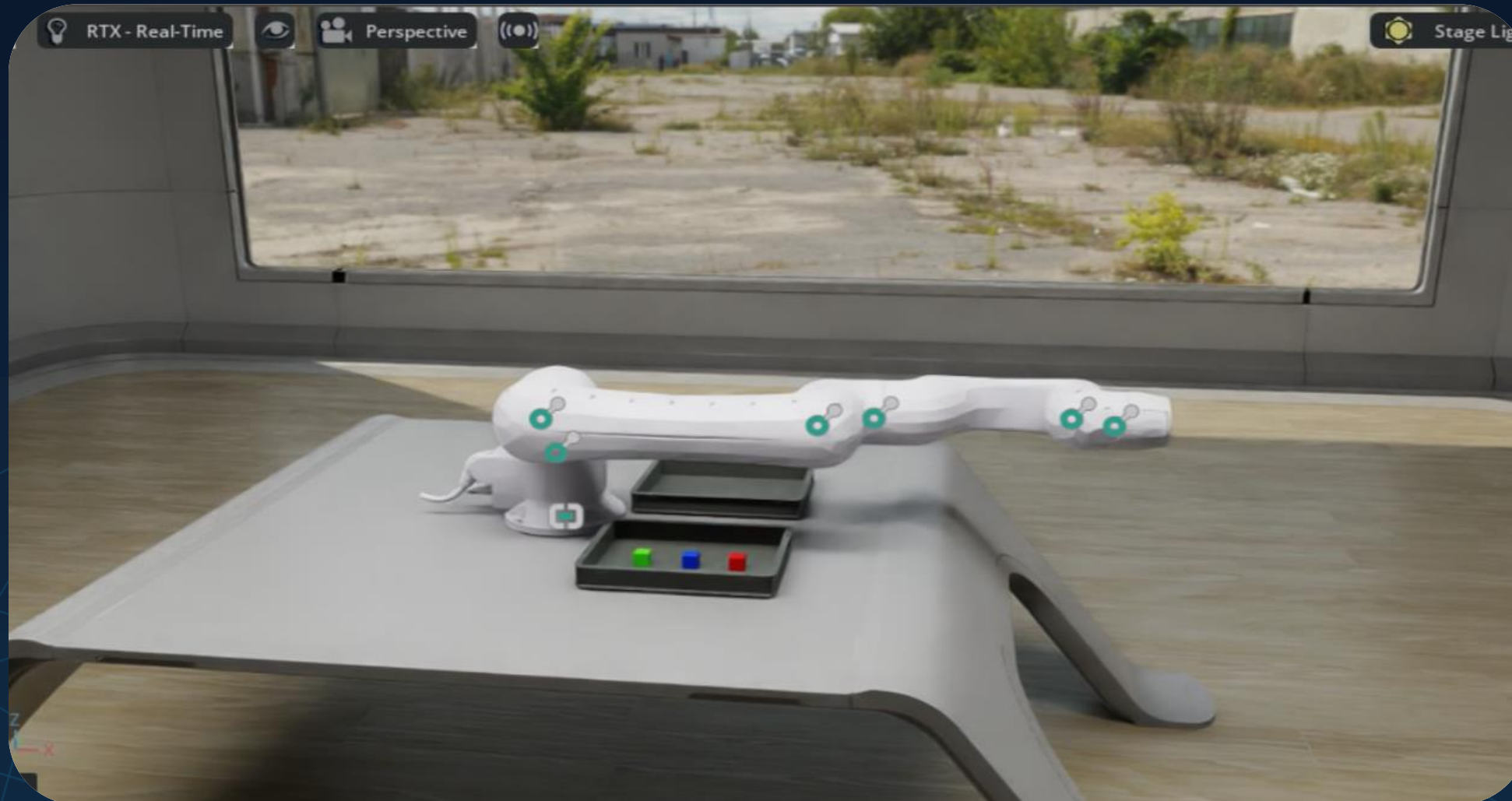
How-to: Run Standalone Scripts

Shutdown Isaac Sim and open the Workshop space in VS Code

```
$ [your Isaac installation root]/python.sh*  
./Day2/02_PickPlace/KUKA_iisy11_robotiq140/pick_up_example.py
```



Assemble Your Scene



Pick & Place Phases

ee... end_effector

- Phase 0: Move above the cube center at the 'end_effector_initial_height'.
- Phase 1: Move *ee* down to encircle the target cube
- Phase 2: Wait for Robot's inertia to settle.
- Phase 3: Close grip.
- Phase 4: Move *ee* up again, keeping the grip tight (lifting the block).
- Phase 5: Move the *ee* toward the goal xy, keeping the height constant.
- Phase 6: Move *ee* vertically toward goal height at the 'end_effector_initial_height'.
- Phase 7: Loosen the grip.
- Phase 8: Move *ee* up again at the 'end_effector_initial_height'
- Phase 9: Move *ee* towards the old xy position.

Save Scene

- Use the USDA format
- Inspect the structure and *payloads*

Documentation



<https://docs.isaacsim.omniverse.nvidia.com/4.5.0/py/index.html>

Online or Local
(latter might not be available)

```
$ cd isaacsim/docs
$ python3 -m http.server 9999
```


Until now:

- Debugging Tools:
 - Console, VS Code, UI
- Scene Architecture
 - Build your Scenario: Environment, Robot, Gripper, Pick & Place

10 min Break

To be continued:

- Motion Planning & Motion Learning
- Materials
- Movie Capture

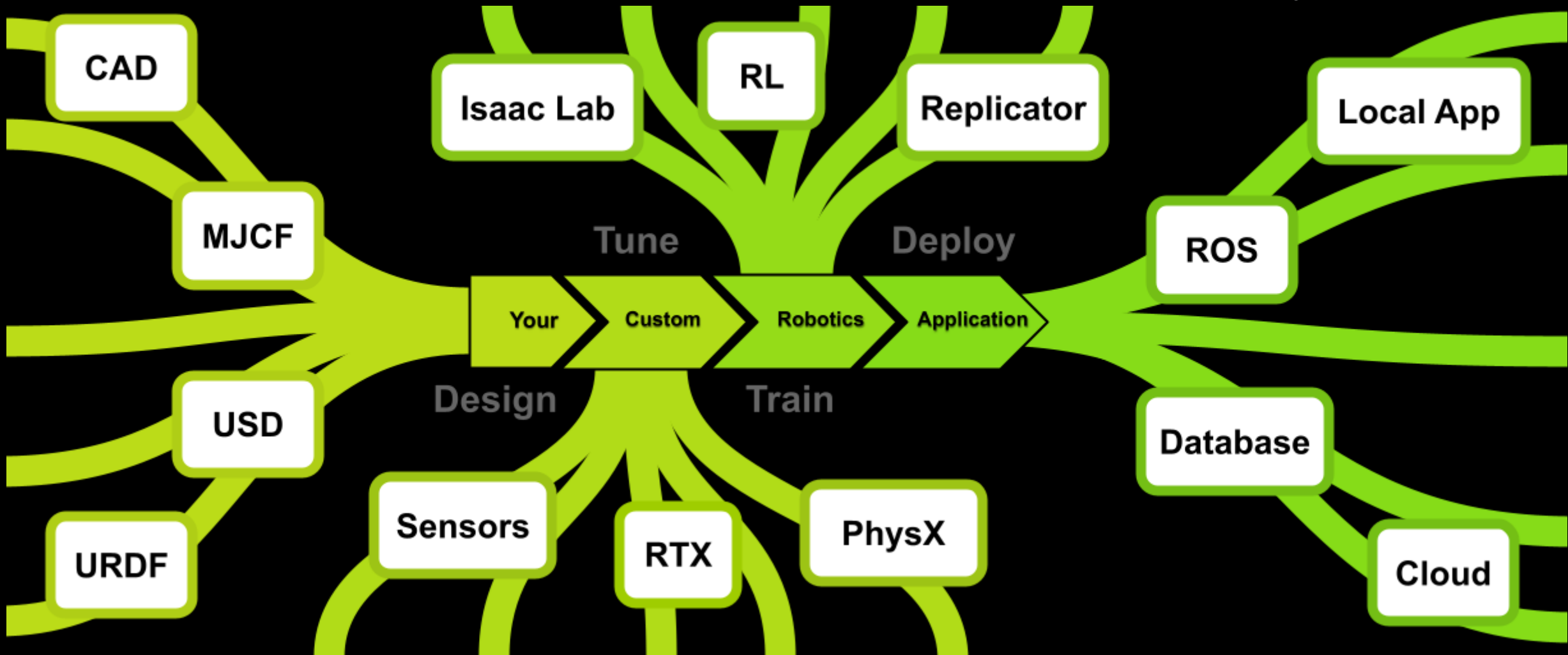
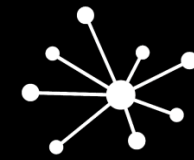
Motion Planning Libraries

- Lula
 - Kinematics Solver (fast-forward and inverse kinematics)
 - Trajectory Generator (using one or more planners)
 - Global Planners (e.g. RRT-Connect, JT-RRT)
 - Local Planners (e.g. RMPflow)
 - CPU-based
- CuMotion
 - Similar capabilities as Lula
 - Supports real-time control
 - Builds upon CuRobo (CUDA-accelerated)

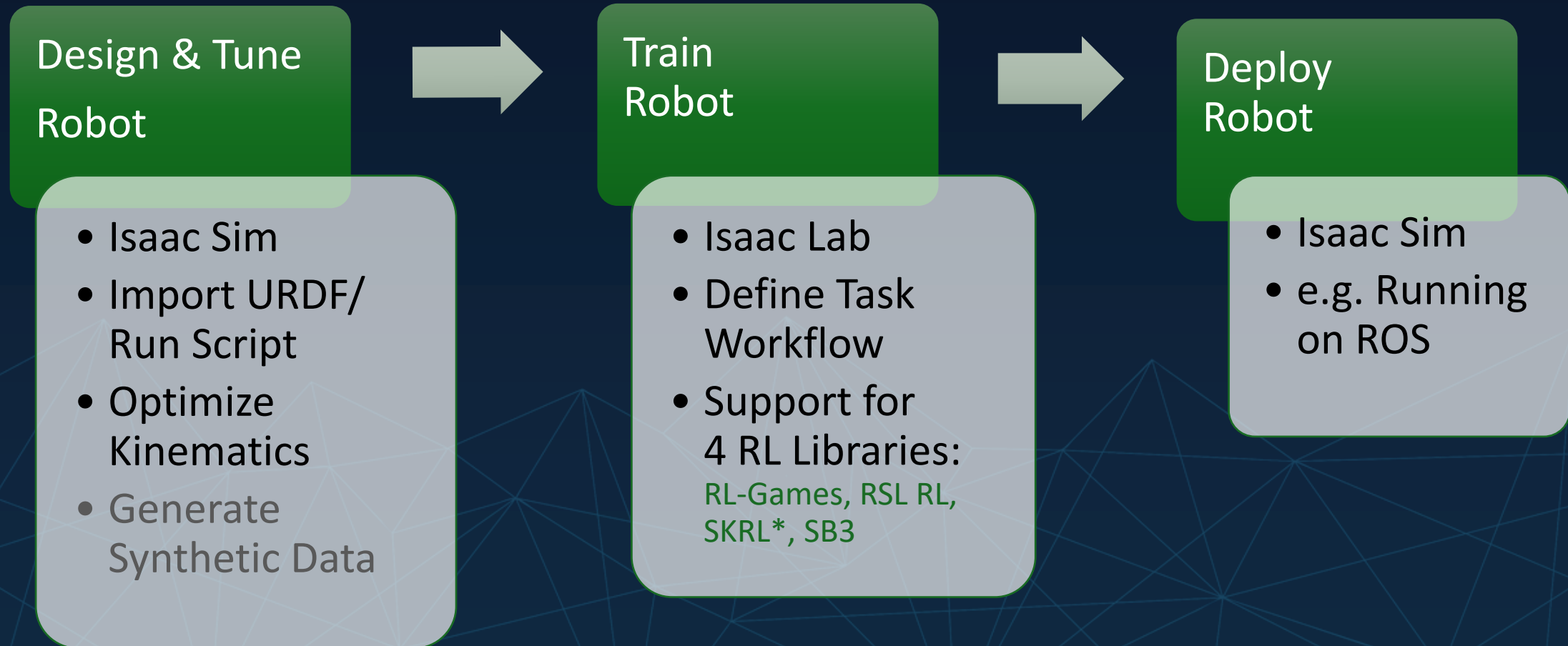
⚠ Lula algorithms require a *robot_description.yaml*. It contains active and fixed joints, default joint positions, collision spheres and more. This format will be superseded by the XRDF-format.

Prerequisites for CuMotion with RMP

- An URDF file, used for specifying robot kinematics as well as joint and link names. Position limits for each joint are also required.
- An (X)RDF file which can be generated using the Lula Robot Description Editor UI tool. (configure drives, robot density and collision meshes)
- A RMPflow configuration file in YAML format, containing parameters for all enabled RMPs.*

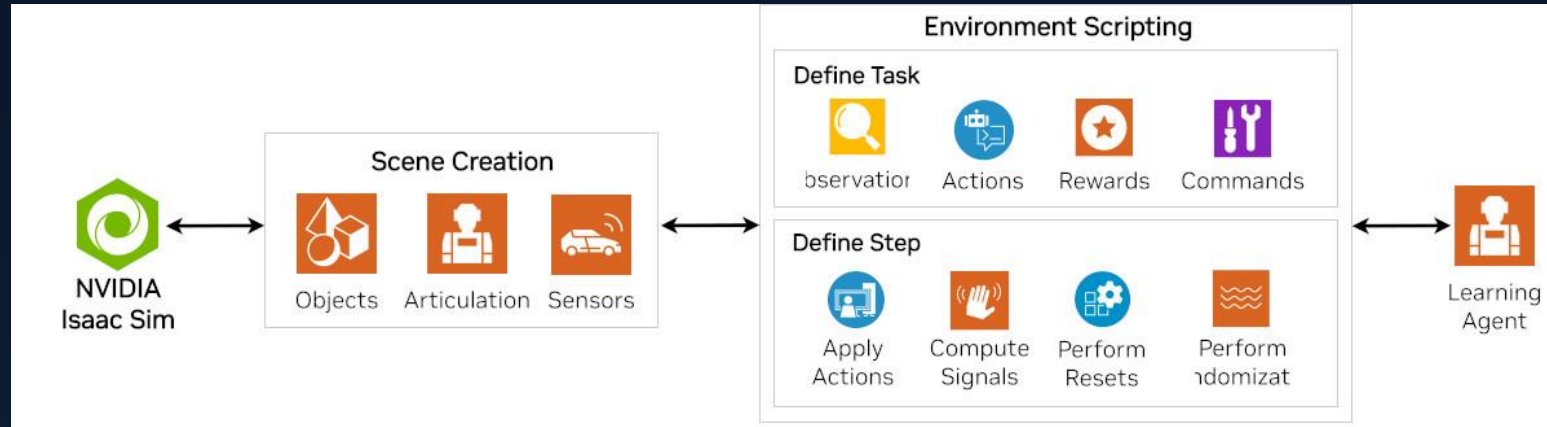


IsaacLab Workflow – High Level



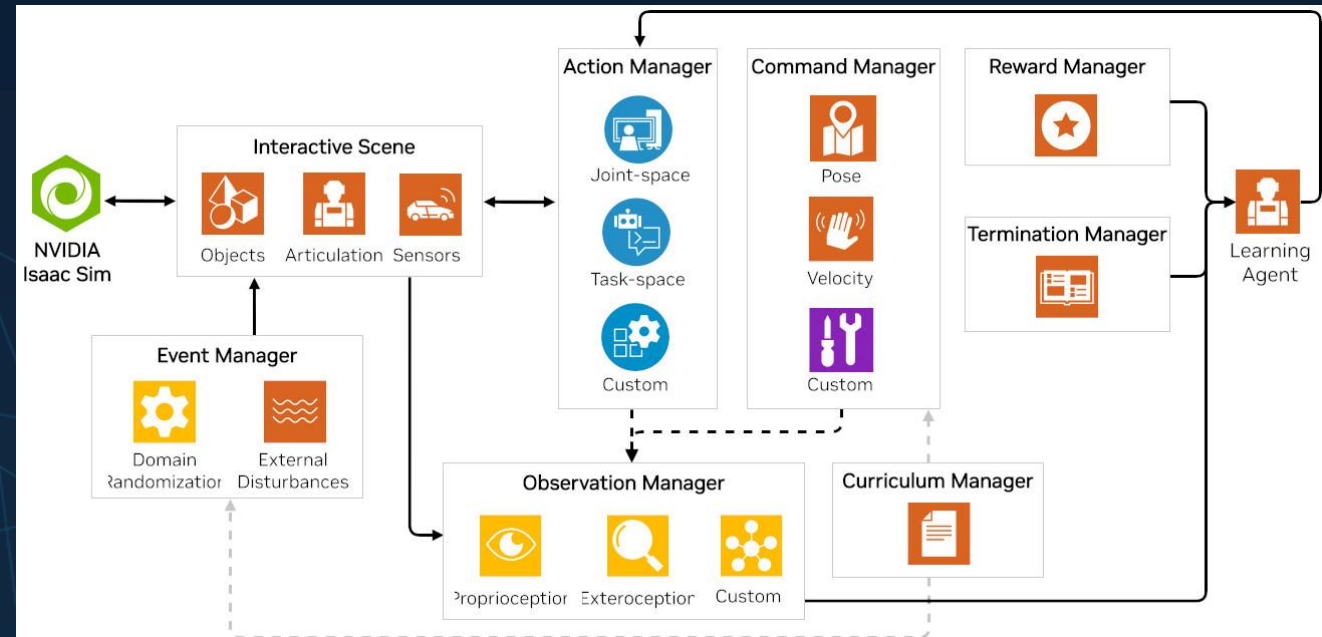


Direct RL Workflow



Task Workflows in IsaacLab

Manager-based RL Workflow



Integrating ML Models

- Training models on synthetic data generated in Isaac Sim or COSMOS
- Foundation model as perception or reasoning module within the Isaac Lab workflow
- Isaac Lab RL policy optimization
- Groot N1:
 - Requires Trajectory Data, Video
 - <https://github.com/NVIDIA/Isaac-GROOT/blob/main/media/model-architecture.png>
- $\pi 0.5$
 - Sensory Data
 - <https://pi.website/blog/pi05>
- OCTO
 - Generalist Robotic Policy
 -  [Octo: An Open-Source Generalist Robot Policy](#)

How to generate Trajectory Data

- Classic: Real Robot „imitation learning“
- Trajectory generators
- Applied RL policies
- Teaching sim. Robot with imitation learning

Until now:

- Debugging Tools:
 - Console, VS Code, UI
- Scene Architecture
 - Build your Scenario: Environment, Robot, Gripper, Pick & Place
- Motion Planning & Motion Learning

5 min Break

To be continued:

- Materials
- Movie Capture

Materials & physics Materials

Materials:

- Color Map
- Reflectivity
- Absorption
- Many others and mixtures/ Shaders

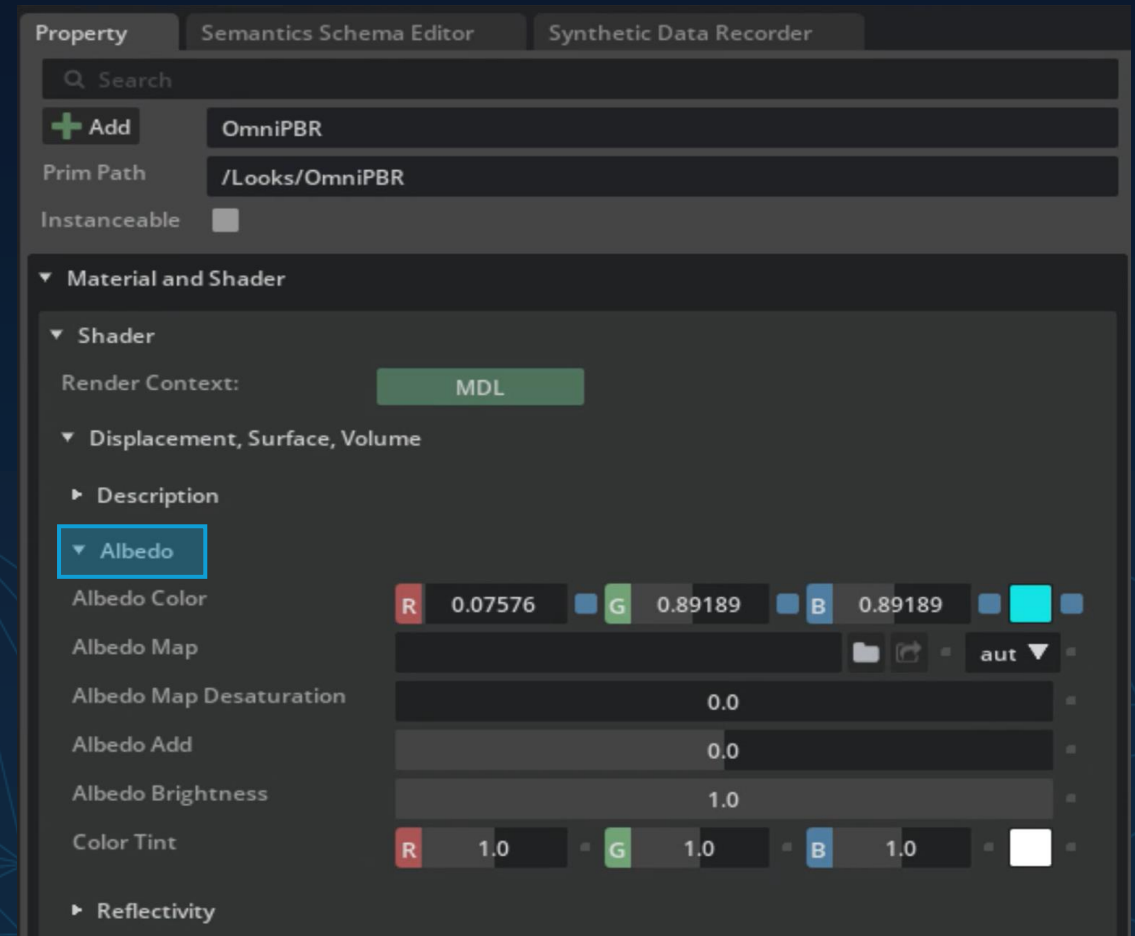
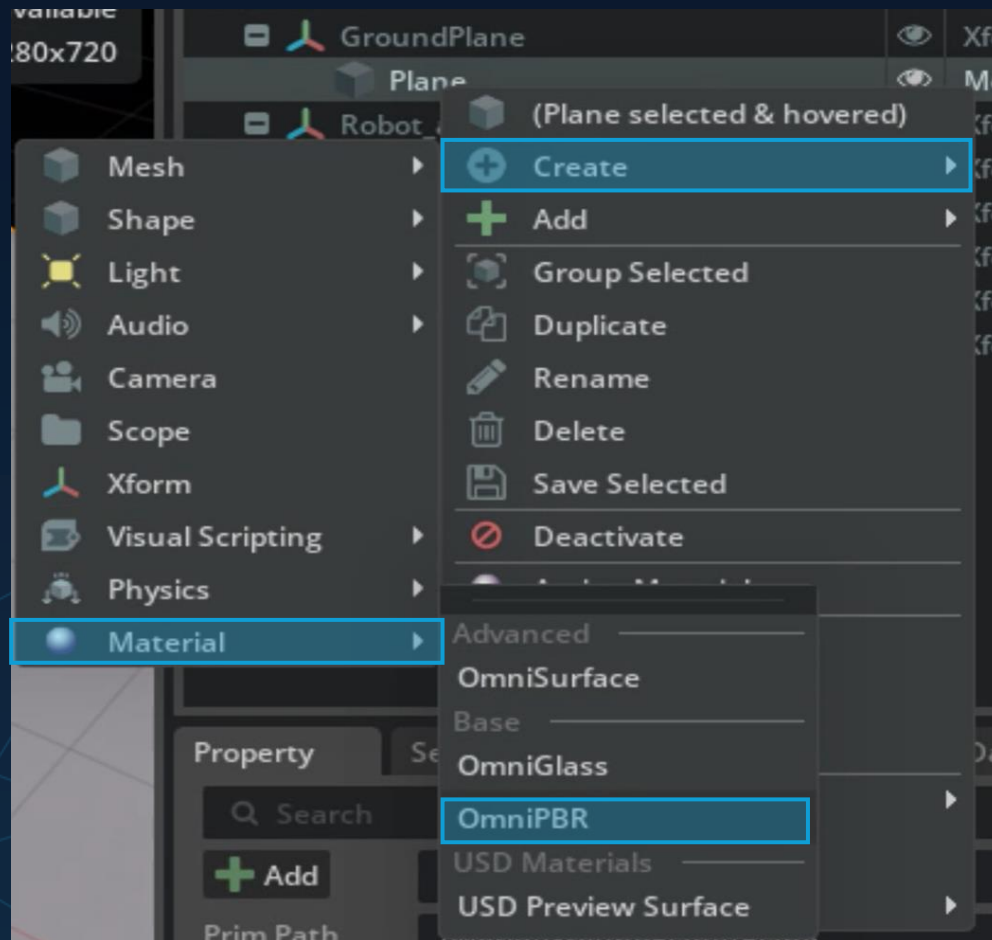


Physics Materials:

- Static Friction
- Dynamic Friction
- Restitution
- Density



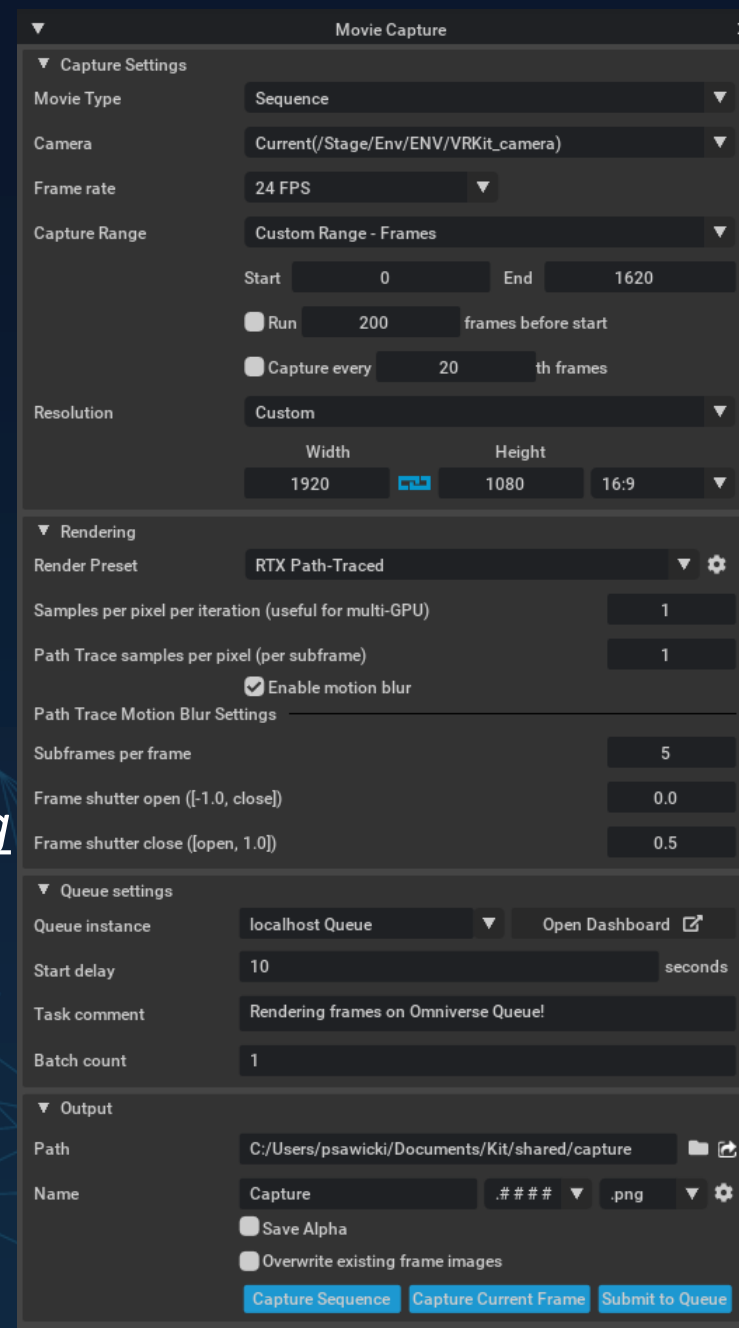
How-to: Create a Material





Movie Capture Tool

- Navigate to Window -> Extensions Manager using the top-level menu
- Search *Movie Capture* in the text field to reveal *omni.kit.window.movie_maker*.
- Click the toggle to enable the extension.
- Repeat for Extension *omni.videoencoding* (Search *Encode*)
- Open the extension in rendering -> Movie Capture

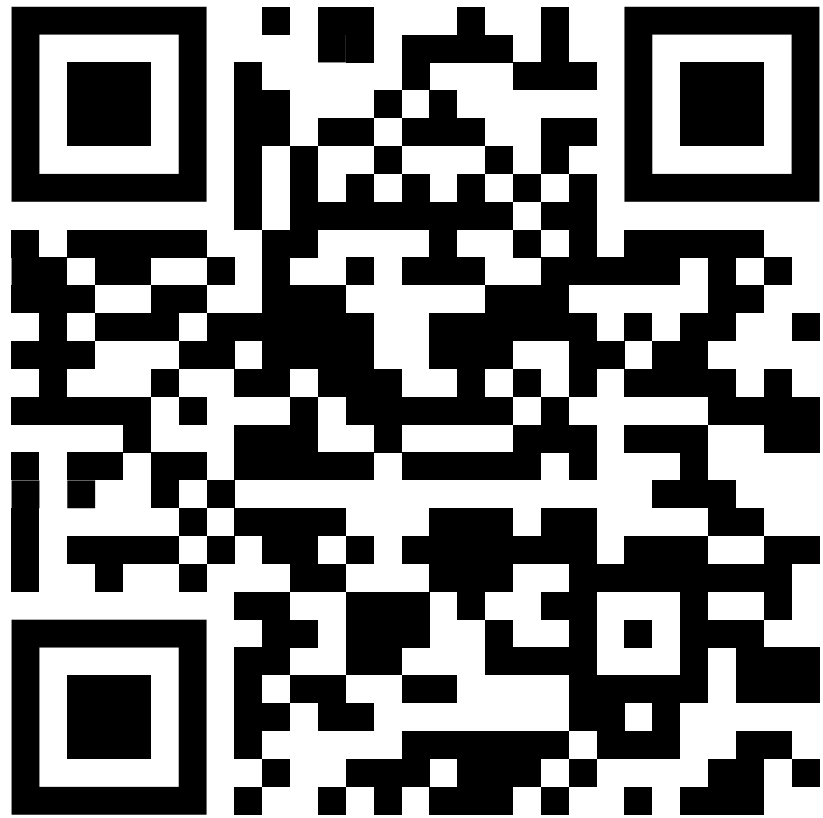




Movie Capture + simple Animation



Feedback Workshop



For further questions mail us:
fabian.fichtl@hs-kempten.de
julian.zuern@hs-kempten.de

Ressources & further Courses

Omniverse Glossary

- [Omniverse Glossary of Terms — Omniverse Developer Workstations](#)

USD

Payload vs Reference:

<https://forums.developer.nvidia.com/t/usd-payload-vs-reference-vs/252658>

Joints

- https://nvidia-omniverse.github.io/PhysX/physx/5.3.0/_downloads/6acf3afb8f69452757e0e766b5a22978/implicitDrives.pdf

URDF Import

- https://docs.isaacsim.omniverse.nvidia.com/4.5.0/robot_setup/ext_isaacsim_asset_importer_urdf.html

Gripper

- https://docs.isaacsim.omniverse.nvidia.com/latest/robot_setup/assemble_robots.html
 - https://docs.isaacsim.omniverse.nvidia.com/latest/robot_simulation/ext_isaacsim_robot_surface_gripper.html
 - cobotta demo und anschließend mit iisy
 - https://docs.isaacsim.omniverse.nvidia.com/latest/robot_setup/import_manipulator.html
 - https://docs.isaacsim.omniverse.nvidia.com/latest/manipulators/manipulators_configure_rmpflow_denso.html#isaac-sim-app-tutorial-configure-rmpflow-denso
- https://docs.isaacsim.omniverse.nvidia.com/latest/robot_setup/grasp_editor.html

Roboter konfiguration

- https://docs.omniverse.nvidia.com/isaacsim/latest/advanced_tutorials/tutorial_configure_rmpflow_denso.html
- Articulation Root
-> Mobil oder stationär
- https://docs.omniverse.nvidia.com/extensions/latest/ext_physics/articulations.html

Movelt

- <https://moveit.picknik.ai/main/index.html>
- <https://moveit.picknik.ai/main/doc/concepts/concepts.html>

Cumotion

- <https://nvidia-isaac-ros.github.io/concepts/manipulation/index.html#nvidia-cumotion>
- Isaac Manipulator consists of a set of components and reference workflows for advanced perception-driven manipulation. These components include state-of-the-art packages for object detection and object pose estimation, as well as obstacle-aware motion generation.

Task Space Controller

- https://isaac-sim.github.io/IsaacLab/main/source/tutorials/05_controllers/run_diff_ik.html

Isaac Lab

- Policy Deployment

[Deploying Isaac Lab Policies in Isaac Sim | Isaac Lab Office Hours](https://cdn.discordapp.com/attachments/1379872643620278484/1379925750668591174/Deploying_Policies_in_Isaac_Sim.pdf?ex=6843fdc0&is=6842ac40&hm=626082febe6a804dcc87c175a523c54913398487f5815a02126293b5eb9bff24&)

https://cdn.discordapp.com/attachments/1379872643620278484/1379925750668591174/Deploying_Policies_in_Isaac_Sim.pdf?ex=6843fdc0&is=6842ac40&hm=626082febe6a804dcc87c175a523c54913398487f5815a02126293b5eb9bff24&

Self paced Courses:

- https://learn.nvidia.com/courses/course?course_id=course-v1:DLI+S-OV-31+V1&unit=block-v1:DLI+S-OV-31+V1+type@vertical+block@bd7da7c3339a42c69f073ca845778391
- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-OV-31+V1
- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-OV-15+V1
- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-OV-27+V1
- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-OV-29+V1
- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-OV-20+V1
- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-OV-28+V1
- <https://academy.nvidia.com/en/course/nvidia-ai-enterprise-administration/?cm=55144>